Laporan Tugas Besar 1
IF3170 Inteligensi Artifisial

Local Search



Disusun oleh:
G59 LazyService

Adiel Rum	10123004
Christopher Justin Kurniawan	13323032
Buege Mahara Putra	13523037

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
Bandung
2026

Daftar Isi

1. Deskripsi Persoalan **2**
1.1. Entitas dan Atribut 2
1.2. Fungsi Objektif 2
1.3. Constraint 3
1.4. Ruang State dan Move 3
1.5. Batas Atas Global Optimum 3
2. Pembahasan **4**
2.1. Representasi dan Komponen Umum 4
2.1.1. Representasi State 4
2.1.2. Inisialisasi Acak 5
2.1.3. Neighbor dan Successor 5
2.1.4. Pemeriksaan Constraint 6
2.2. Implementasi Hill Climbing 9
2.2.1. Steepest Ascent Hill Climbing 9
2.2.2. Stochastic Hill Climbing 15
2.2.3. Random Restart Hill Climbing 22
2.2.4. Sideways Move Hill Climbing 29
2.3. Implementasi Simulated Annealing 37
2.4. Implementasi Genetic Algorithm 45
2.5. Perbandingan Antar Algoritma 50
2.6. Bonus: Fungsi Objektif Urutan Bongkar (LIFO) 53
2.6.1. Definisi 53
2.6.2. Implementasi 54
2.6.3. Hasil dan Analisis 56
3. Kesimpulan dan Saran **61**
3.1. Kesimpulan 61
3.2. Saran 61
Daftar Pustaka **62**
Lampiran **63**
A. Pembagian Tugas dan Repo Github 63
B. Input Multi Truk dan Satu Truk 63
C. Bonus yang Dikerjakan 64
Form Penggunaan AI **65**

1. Deskripsi Persoalan

Tugas Besar 1 mata kuliah IF3170 Inteligensi Artifisial ini berfokus mengimplementasikan *local search* untuk menyelesaikan permasalahan berupa studi kasus optimasi pengisian paket (*3D bin packing*) pada perusahaan logistik *Mejiro Express*. Tugas ini bertujuan untuk mencari konfigurasi penataan paket yang paling optimal pada peti kemas truk dengan memaksimalkan total nilai (*value*) barang, serta wajib memenuhi berbagai batasan seperti batas kapasitas berat truk, aturan penempatan barang pecah-belah, dan ketentuan fisik agar kardus tidak melayang atau menembus dinding truk.

Algoritma yang digunakan adalah *steepest ascent hill climbing, hill climbing with sideways, random restart hill climbing, stochastic hill climbing, simulated annealing,* dan *genetic algorithm.*

1.1. Entitas dan Atribut

Terdapat dua entitas, yaitu paket dan truk. Seluruh paket berbentuk balok, dan seluruh koordinat serta dimensi berupa bilangan bulat. **Tabel 1.1.1** merangkum atribut kedua entitas.

Tabel 1.1.1. Entitas dan atribut persoalan.

Entitas	Atribut	Keterangan
Paket	id	Pengenal unik paket.
	dimensions (w, l, h)	Lebar, panjang, dan tinggi paket.
	orientation	Penetapan sisi paket w, l, h ke sumbu x, y, z. Terdapat enam kemungkinan orientasi.
	value	Nilai paket, yang dijumlahkan pada fungsi objektif.
	weight	Berat paket.
	is_fragile	Penanda paket pecah-belah.
	eta	Perkiraan waktu kedatangan paket (dalam sisa hari).
Truk	dimensions (W, L, H)	Lebar, panjang dan tinggi ruang muat.
	max_capacity	Kapasitas berat maksimum.

1.2. Fungsi Objektif

Tujuan persoalan ini adalah memaksimalkan total *value* paket yang berada di dalam truk. Paket di luar truk tidak dihitung terhadap fungsi objektif. Pada persoalan multi-truk, nilai yang dimaksimalkan adalah total nilai seluruh paket di semua truk, dan satu paket hanya dapat berada pada satu truk.

1.3. Constraint

Sebuah konfigurasi dianggap layak apabila memenuhi kelima syarat berikut untuk setiap truk.

1. **C1: batas ruang muat.** Seluruh bagian paket harus berada di dalam peti kemas, sehingga paket tidak boleh menembus dinding.
2. **C2: tidak bertabrakan.** Dua paket pada truk yang sama tidak boleh menempati ruang yang sama.
3. **C3: tidak melayang.** Alas paket harus memiliki penopang. Paket yang berada di lantai selalu tertopang, sedangkan paket lain harus bertumpu pada bagian atas paket lain. Agar paket dianggap tidak melayang, cukup satu titik pada alas paket tersebut yang butuh ditopang.
4. **C4: barang pecah-belah tidak boleh menopang.** Paket fragile tidak boleh berada di bawah paket lain, sehingga tidak boleh ada paket yang bertumpu di atasnya.
5. **C5: kapasitas.** Total berat paket di dalam sebuah truk tidak boleh melebihi kapasitas truk tersebut.

Posisi paket ditentukan oleh sudut kiri-bawah-belakangnya, dan ukurannya pada tiap sumbu ditentukan oleh orientasi yang dipilih. Sebuah paket dengan dimensi $1 \times 2 \times 3$, misalnya, dapat menempati ruang $3 \times 2 \times 1$, $2 \times 1 \times 3$, dan seterusnya sesuai orientasinya.

1.4. Ruang State dan Move

State adalah pemetaan lengkap semua paket, yaitu truk, koordinat, dan orientasi setiap paket, sehingga mencakup keadaan di dalam maupun di luar truk. Inisialisasi dilakukan secara acak. Tiga jenis move yang diizinkan untuk berpindah ke neighbor adalah:

1. menukar dua paket, baik keduanya di dalam truk, di dalam dan di luar, ataupun lintas truk,
2. memindahkan sebuah paket ke koordinat (dan truk) yang berbeda, dan
3. merotasi sebuah paket sebesar 90° terhadap sumbu x, y, dan z.

Successor yang melanggar C1 sampai C5 tidak dianggap sebagai neighbor yang valid. Seluruh algoritma hanya bergerak di antara state yang valid.

1.5. Batas Atas *Global Optimum*

Nilai optimum global tidak diketahui secara pasti karena persoalan ini tergolong NP-*hard*. Sebagai pembanding, dihitung batas atas U dengan menyederhanakan persoalan menjadi pemilihan paket yang hanya dibatasi total berat dan total volume, dengan mengabaikan susunan paket, penopang, dan pembagian ke tiap truk. Nilai U dihitung dengan dynamic programming knapsack 0/1. Karena penyederhanaan ini hanya memperlonggar persoalan, nilai optimum tidak mungkin melebihi U. Oleh karena itu, rasio nilai solusi terhadap U merupakan batas bawah dari rasio terhadap optimum sebenarnya.

2. Pembahasan

2.1. Representasi dan Komponen Umum

2.1.1. Representasi State

Kelas `State` menyimpan konfigurasi lengkap dalam tiga pemetaan dengan kunci id paket, yaitu `truck_of` (indeks truk atau None jika di luar truk), `positions` (sudut kiri-bawah-belakang (*x, y, z*)), dan `orientations` (permutasi (*w, l, h*) pada sumbu *x, y, z*). Agar evaluasi efisien, `State` juga menyimpan tiga *cache* yang selalu disinkronkan oleh metode `place` dan `rotate`, yaitu total nilai (`value`), total berat per truk (`truck_weights`) dan kotak (*x, y, z, dx, dy, dz*) setiap paket yang berada di dalam truk (`boxes`). Dengan ini, fungsi objektif dan pemeriksaan C5 hanya berbiaya $O(1)$. Data persoalan yang tidak berubah (`Problem`) dipakai bersama seluruh *state*, sehingga `copy` hanya menyalin ketiga pemetaan tersebut.

Tabel 2.1.1.1 merangkum metode State yang dipakai pada bagian-bagian berikutnya. Metode pembantu kecil yang tidak dirujuk di tempat lain tidak dicantumkan.

Tabel 2.1.1.1. Metode `State`.

Metode	Fungsi
`copy`	Menyalin ketiga pemetaan dan cache. `Problem` dipakai bersama.
`place`	Menempatkan atau mengeluarkan satu paket, sekaligus menyinkronkan `value`, `truck_weights`, dan `boxes`.
`rotate`	Memutar satu paket 90° pada sumbu x, y, atau z dan memperbarui `boxes`.
`inside, outside`	Daftar id paket yang berada di dalam suatu truk (atau truk mana pun) dan yang berada di luar semua truk.
`objective`	Total nilai paket di dalam truk, dibaca dari *cache* `value`.
`within_bounds, collides`	Memeriksa satu paket terhadap batas truk (C1) dan terhadap tumpang tindih dengan paket lain (C2).
`is_supported`	Memeriksa penopang satu paket, termasuk aturan paket *fragile* (C3 dan C4).
`resting_on`	Daftar paket yang alasnya tepat berada di atas paket tertentu.
`valid`	Pemeriksaan penuh seluruh *constraint* pada seluruh paket.
`valid_after_change`	Pemeriksaan inkremental setelah paket tertentu berubah, setara dengan `valid` untuk *state* asal yang layak.

Metode	Fungsi
`z_levels`	Ketinggian *z* yang mungkin bagi satu paket, baik di lantai truk maupun puncak paket lain.
`candidate_positions`	Seluruh posisi (*x, y, z*) kandidat bagi satu paket pada satu truk, memakai `z_levels`.
`random_candidate_position`	Satu posisi acak dari kandidat yang sama, selain posisi saat ini, tanpa membangun seluruh daftarnya.
`apply`	Menerapkan satu *move* pada salinan *state*. Mengembalikan `None` jika melanggar *constraint*.
`all_moves`	Seluruh *move* yang mungkin, belum disaring kelayakannya.
`all_successors`	*Successor* layak hasil `apply` untuk semua `all_moves`.
`random_successor`	Satu *successor* layak dari *move* yang dipilih acak.
`random_initial`	Pembangkit *state* awal acak yang layak.

Terdapat tiga jenis *move* pada `apply`. *Swap* menukar truk dan posisi dua paket, dengan syarat sedikitnya satu paket berada di dalam truk. *Move* memindahkan satu paket ke posisi kandidat pada suatu truk. *Rotate* memutar satu paket yang berada di dalam truk pada salah satu sumbu.

2.1.2. Inisialisasi Acak

`State.random_initial` memulai dari seluruh paket di luar truk dengan orientasi acak. Paket diproses dalam urutan acak. Dengan peluang 0.5, sebuah paket dicoba dimasukkan ke truk acak pada posisi kandidat acak, dan posisi pertama yang memenuhi seluruh *constraint* dipakai. Cara ini menjamin state awal valid sekaligus beragam antarpercobaan.

2.1.3. *Neighbor* dan *Successor*

Fungsi `State.apply` menerapkan satu move (`swap`, `move`, atau `rotate`) pada salinan *state* dan mengembalikan `None` jika hasilnya melanggar *constraint*. Dua cara membangkitkan *successor* dipakai sesuai kebutuhan algoritma:

a. `all_successors` menyusun seluruh *move* yang mungkin lalu menyaring yang valid. Fungsi ini dipakai oleh *steepest ascent, sideways move,* dan *random restart* untuk memilih *neighbor* bernilai tertinggi, dengan pemilihan acak jika terdapat nilai tertinggi yang sama.
b. `random_successor` memilih secara acak jenis *move* dan parameternya hingga ditemukan *successor* yang valid. Fungsi ini dipakai oleh *stochastic hill climbing, simulated annealing,* dan mutasi pada *genetic algorithm*.

Kode 2.1.3.1. Penerapan satu *move* pada `State`.

```
def apply(self, move: Move) -> State | None:
    successor = self.copy()
    changed: tuple[str, ...]

    if len(move) == 4: # move = "move"
        _, package_id, truck_index, new_position = move
        changed = (package_id,)
        successor.place(package_id, truck_index, new_position)
    elif move[0] == "swap":
        _, package_id_1, package_id_2 = move
        changed = (package_id_1, package_id_2)
        successor.place(
            package_id_1, self.truck_of[package_id_2],
            self.positions[package_id_2]
        )
        successor.place(
            package_id_2, self.truck_of[package_id_1],
            self.positions[package_id_1]
        )
    else:
        _, package_id, axis = move
        changed = (package_id,)
        successor.rotate(package_id, axis)

    if successor.valid_after_change(changed, self):
        return successor
    return None
```

Sebuah posisi hanya layak apabila z = 0 atau z = tinggi puncak paket lain, karena selain itu paket pasti melayang (C3). z_levels hanya membangkitkan ketinggian tersebut. Reduksi ini tidak menghilangkan satu pun posisi layak, tetapi menurunkan jumlah kandidat secara drastis dari semua z bulat menjadi sejumlah ketinggian puncak yang ada.

2.1.4. Pemeriksaan *Constraint*

Pemeriksaan penuh (`State.valid`) memeriksa C1, C2, C3, dan C5 untuk seluruh paket, sedangkan C4 ditegakkan di dalam `is_supported`. Pemeriksaan penuh berbiaya $O(n^2)$ terhadap jumlah paket, sehingga terlalu mahal jika dipanggil untuk setiap successor.

Kode 2.1.4.1. Pemeriksaan penopang (C4).

```
def is_supported(self, package_id: str) -> bool:
    package_box = self.box(package_id)
    z = package_box[2]
    if z == 0:
        return True

    supported = False
    for below_id, box_below in self.other_boxes_in_truck(package_id):
        if box_below[2] + box_below[5] == z and bases_overlap(package_box,
            box_below):
            if self.packages[below_id].is_fragile:
                return False
            supported = True
    return supported
```

Karena *state* asal selalu layak, `valid_after_change` cukup memeriksa bagian yang dapat terdampak oleh *move*: (i) paket yang berubah beserta kapasitas truk penerimanya, dan (ii) paket yang pada *state* asal bertumpu di atas paket yang berpindah, karena penopangnya hilang. Jika paket yang berubah bersifat *fragile*, tidak boleh ada paket lain yang bertumpu di atasnya. *Constraint* lain tidak tersentuh oleh *move* sehingga tetap terpenuhi. Pemeriksaan ini setara dengan valid untuk *state* asal yang layak, tetapi hanya menyentuh sebagian kecil paket.

Kode 2.1.4.2. Pemeriksaan inkremental setelah move.

```
def valid_after_change(self, changed: Sequence[str], before: State | None = None)
-> bool:
    for package_id in changed:
        truck_index = self.truck_of[package_id]
        if truck_index is None:
            continue
        if self.total_weight(truck_index) > self.trucks[truck_index].max_capacity:
            return False
        if not self.within_bounds(package_id) or self.collides(package_id) or not
            self.is_supported(package_id):
            return False
        if self.packages[package_id].is_fragile and self.resting_on(package_id):
            return False

    if before is not None:
        for changed_id in changed:
            if before.truck_of[changed_id] is None:
                continue
```

```
            for package_id in before.resting_on(changed_id):
                if package_id not in changed and not
                   self.is_supported(package_id):
                    return False

    return True
```

2.2. Implementasi Hill Climbing

Algoritma *hill climbing* bergerak dari *current state* ke *neighbor state* yang memiliki fungsi objektif lebih baik (untuk tipe tertentu bisa sama dengan atau lebih kecil dari, tergantung tipe hill climbing yang digunakan). Pencarian akan berhenti ketika telah mencapai jumlah iterasi maksimum atau ketika tidak ditemukan lagi tetangga yang lebih baik, dengan representasi seperti pada **2.1.3. Neighbor dan Successor**. Seluruh successor atau tetangga yang dibangkitkan divalidasi terlebih dahulu terhadap *constraint*. Pada eksperimen akan dilakukan tiga jenis algoritma *hill climbing*.

Berdasarkan eksperimen, didapatkan bahwa algoritma **Random Restart Hill Climbing** lebih sering menemukan solusi lebih optimal untuk persoalan dalam eksperimen ketimbang algoritma *hill climbing* lain.

2.2.1. Steepest Ascent Hill Climbing

Variasi ini membangkitkan seluruh *neighbor state*, beserta fungsi objektifnya, yang mungkin dari *state* saat ini. Algoritma kemudian memilih *neighbor state* dengan fungsi objektif paling tinggi untuk dijadikan sebagai *state* berikutnya. Jika kemudian tidak ditemukan *neighbor state* dengan nilai fungsi objektif lebih tinggi, pencarian dihentikan karena algoritma tidak bisa berpindah ke *state* yang lebih baik.

Kode 2.2.1.1. Kode implementasi dari *Steepest Ascent Hill-Climbing*.

```
class SteepestAscentHC(HillClimbing):
    name = "Steepest Ascent Hill-Climbing"

    def search(self) -> SearchResult:
        initial_state = State.random_initial(self.problem)
        state = initial_state
        history = [state.objective()]

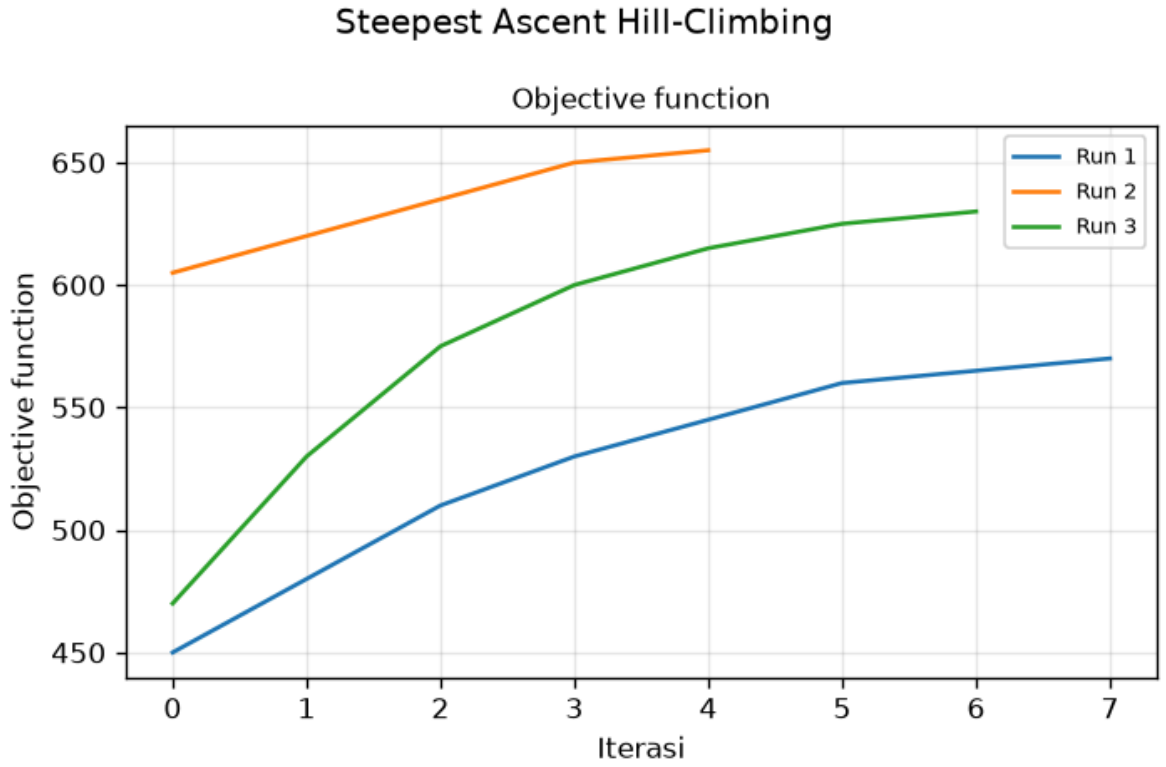
        while True:
            neighbor_value, neighbor = self.best_successor(state)
```

```
            if neighbor_value <= state.objective():
                break
            state = neighbor
            history.append(neighbor_value)

        return SearchResult(initial_state, state, history,
iterations=len(history) - 1)
```

2.2.1.1. Satu Truk

Berdasarkan eksperimen **Gambar 2.2.1.1.1**, setelah menjalankan 3 kali algoritma *steepest ascent hill climbing*, algoritma tidak konsisten dalam mencari solusi global optima karena algoritma hanya melakukan *hill climbing* dan tidak punya mekanisme untuk "menuruni bukit" akibatnya, apabila mulai dari *state* yang buruk, algoritma akan dengan mudah terjebak pada solusi lokal optima tanpa mencapai solusi *global optima*.



Gambar 2.2.1.1.1. Plot objektif terhadap iterasi pada *Steepest Ascent Hill-Climbing*.

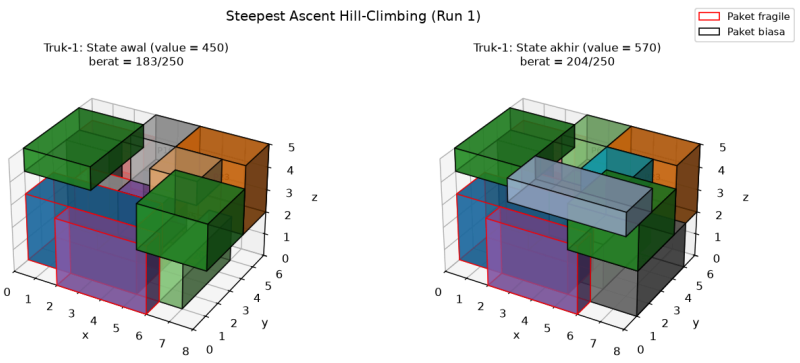
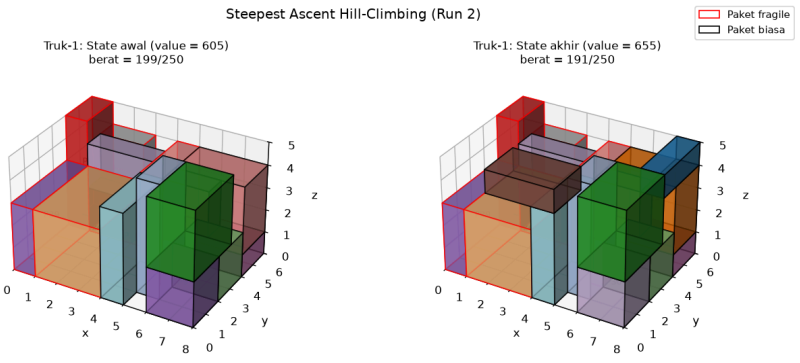
Berdasarkan **Tabel 2.2.1.1.1** terlihat bahwa hasil initial randomization cukup mempengaruhi value akhir dari iterasi. Pada ketiga run ditemukan pola yang konsisten yaitu run dengan value awal yang lebih tinggi mencapai value akhir yang lebih tinggi pada runtime yang lebih rendah. Pola monoton ini

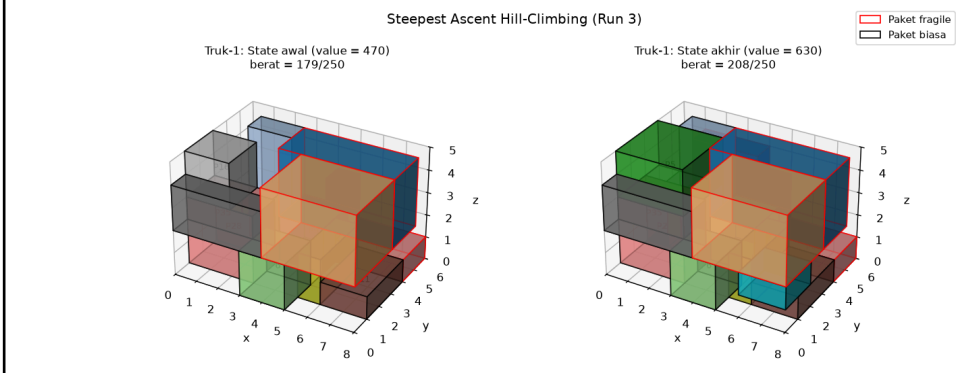
disebabkan karena sifat dari algoritma Steepest Ascent Hill Climbing yang tidak ada aspek diversifikasinya pada tahap pencarian solusi.

Tabel 2.2.1.1.1. Objektif *initial* dan *final state*, dan *runtime* untuk masing-masing *run*.

Run ke-	Value Awal	Value Akhir	Iterasi	Durasi
1	450	570	7	0.12 s
2	605	655	4	0.07 s
3	470	630	6	0.09 s

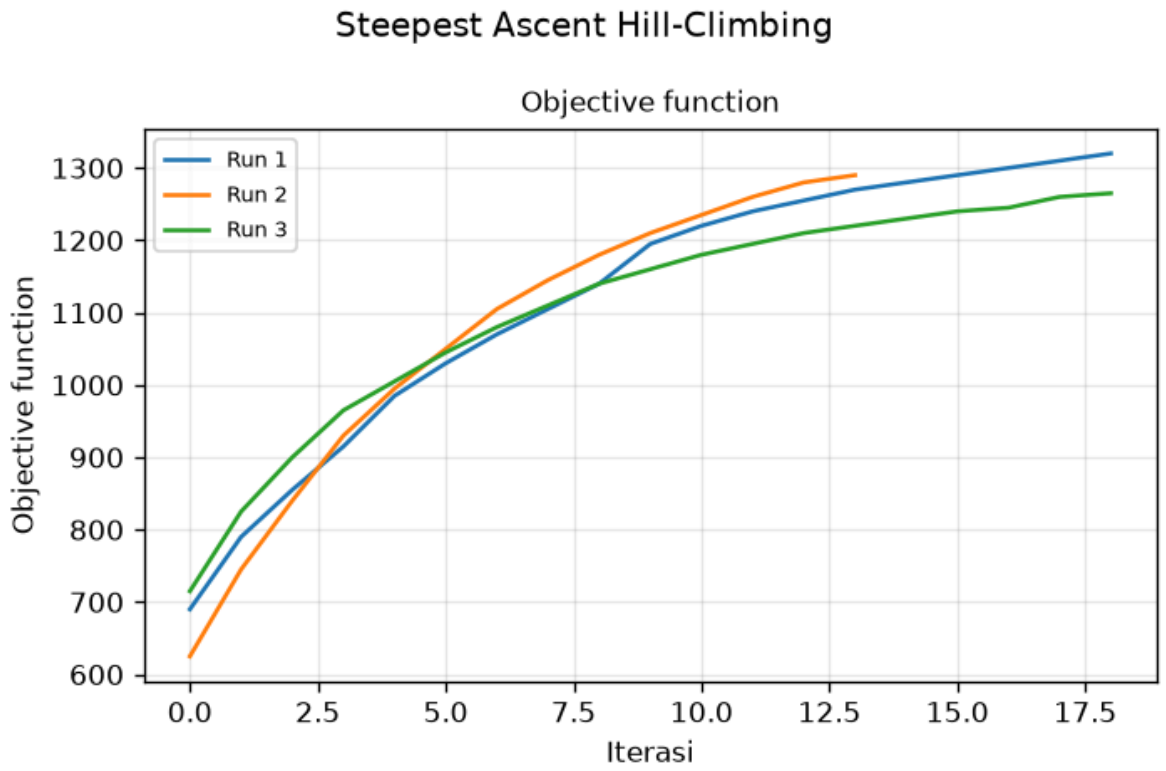
Tabel 2.2.1.1.2. Visualisasi penempatan paket pada truk pada iterasi awal dan akhir algoritma *Steepest Ascent Hill-Climbing*.

Run ke-	Visualisasi Penempatan Paket pada Satu Truk
1	
2	

Run ke-	Visualisasi Penempatan Paket pada Satu Truk
3	

2.2.1.2. Multi Truk

Algoritma kemudian dicoba untuk menyelesaikan persoalan multi-truk hingga mencapai solusi optima dengan spesifikasi paket yang berbeda dari spesifikasi paket untuk satu truk. Didapatkan hasil bahwa pada persoalan multi truk, algoritma *steepest ascent hill climbing* mampu dengan cepat (jumlah iterasi sedikit) mencari solusi optima. Namun solusi yang dihasilkan belum tentu global optima. Pendekatan ini baik apabila diperlukan *immediate* solution karena menghasilkan solusi dengan fungsi objektif cukup baik dalam jumlah iterasi singkat.



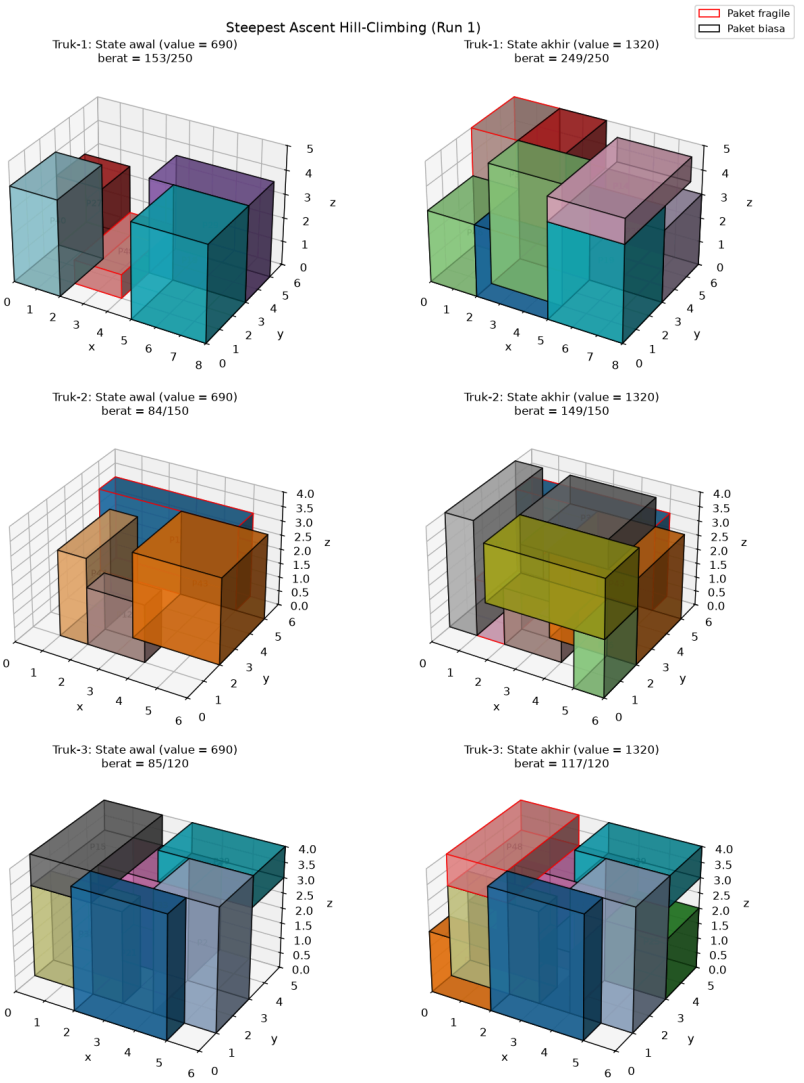
Gambar 2.2.1.2.1 Plot objektif terhadap iterasi pada *Steepest Ascent Hill-Climbing* multi-truk.

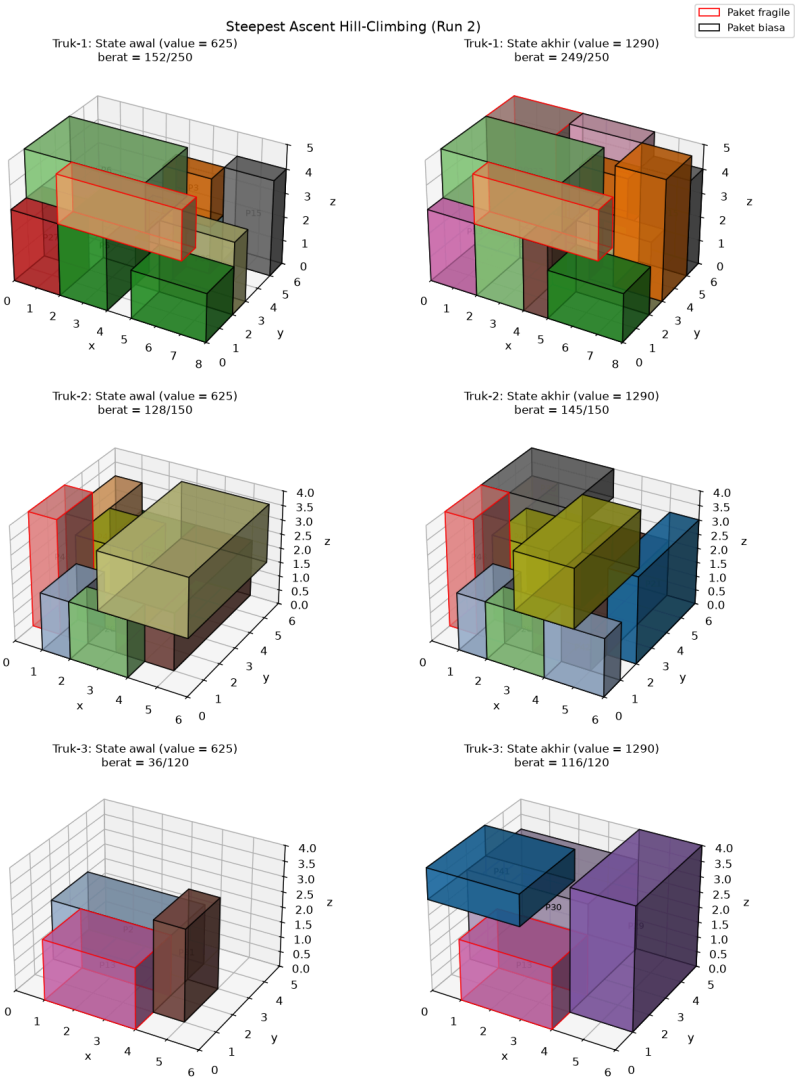
Dapat diamati dari **Tabel 2.2.1.2.1.** Bahwa pola monoton yang diklaim di kasus satu truck, ternyata tidak terjadi di sini, justru pada run ke-3 di mana value awal tertinggi menghasilkan value akhir yang paling rendah pada iterasi yang paling banyak. Hal ini tentu mengamplifikasi kesimpulan bahwa algoritma Steepest Ascent Hill Climbing memang masih suboptimal karena kurangnya mekanisme diversifikasi solusi dalam pencarian solusi yang terdapat pada algoritma Steepest Ascent Lainnya.

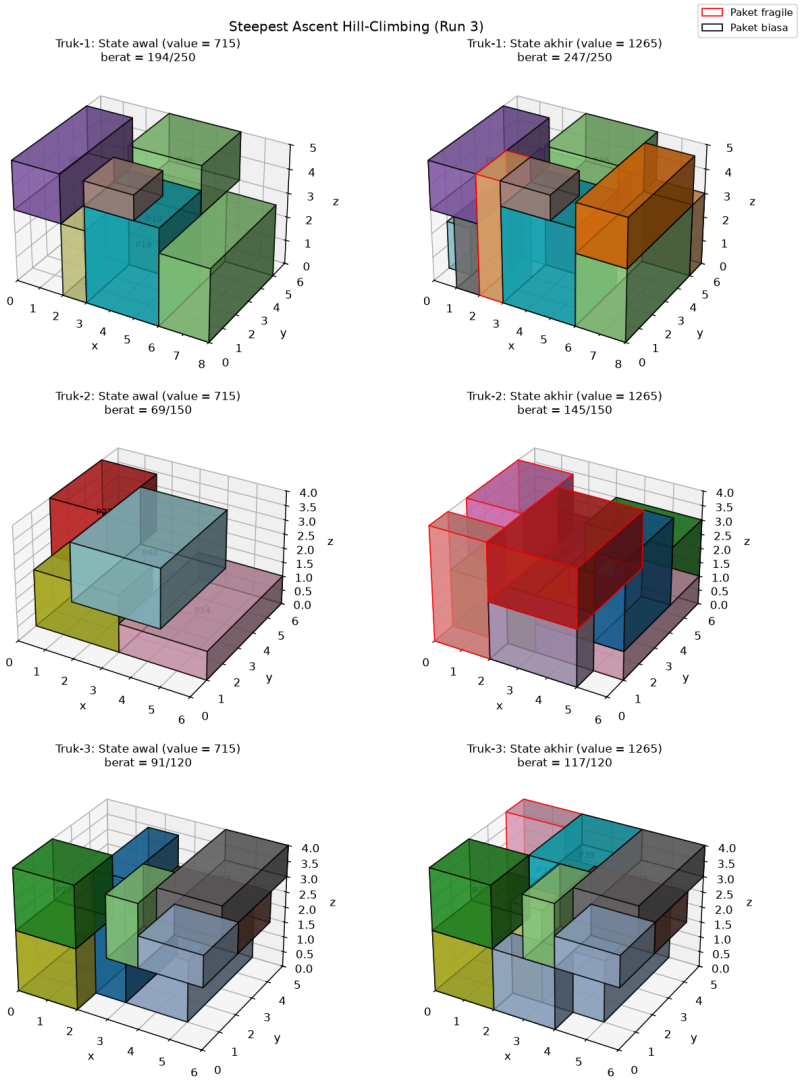
Tabel 2.2.1.2.1. Objektif *initial* dan *final state*, dan *runtime* untuk masing run multi-truk

Run ke-	Value Awal	Value Akhir	Iterasi	Durasi
1	690	1320	18	0.32 s
2	625	1290	13	0.34 s
3	715	1265	18	0.47 s

Tabel 2.2.1.1.2. Visualisasi penempatan paket pada truk pada iterasi awal dan akhir algoritma *Steepest Ascent Hill-Climbing* multi-truk.

Run ke-	Visualisasi Penempatan Paket pada Multi Truk
1	

Run ke-	Visualisasi Penempatan Paket pada Multi Truk
2	

Run ke-	Visualisasi Penempatan Paket pada Multi Truk
3	

2.2.2. Stochastic Hill Climbing

Algoritma ini memilih satu *neighbor state*, beserta fungsi objektifnya, secara acak kemudian dibandingkan dengan *current state*. Apabila fungsi objektif *neighbor state* lebih besar daripada *current state*, state berikutnya langsung berpindah menjadi *neighbor state* tanpa menelusuri seluruh *neighbor state* terlebih dahulu. Hal tersebut membuat iterasi berjalan lebih cepat karena tidak perlu cek *seluruh neighbor state*.

Kode 2.2.2.1. Kode Implementasi dari *Stochastic Hill-Climbing*

```
class StochasticHC(HillClimbing):
    name = "Stochastic Hill-Climbing"
```

```
    def __init__(self, problem: Problem, max_iterations: int) -> None:
        super().__init__(problem)
        self.max_iterations = max_iterations

    def parameter(self) -> dict[str, Any]:
        return {"max_iterations": self.max_iterations}

    def search(self) -> SearchResult:
        initial_state = State.random_initial(self.problem)
        state = initial_state
        history = [state.objective()]

        # Tiap iterasi bangkitin satu successor random, pindah jika lebih baik
        for _ in range(self.max_iterations):
            neighbor = state.random_successor()
            if neighbor.objective() > state.objective():
                state = neighbor
            history.append(state.objective())

        return SearchResult(initial_state, state, history,
                            iterations=self.max_iterations)
```

2.2.2.1. Satu Truk

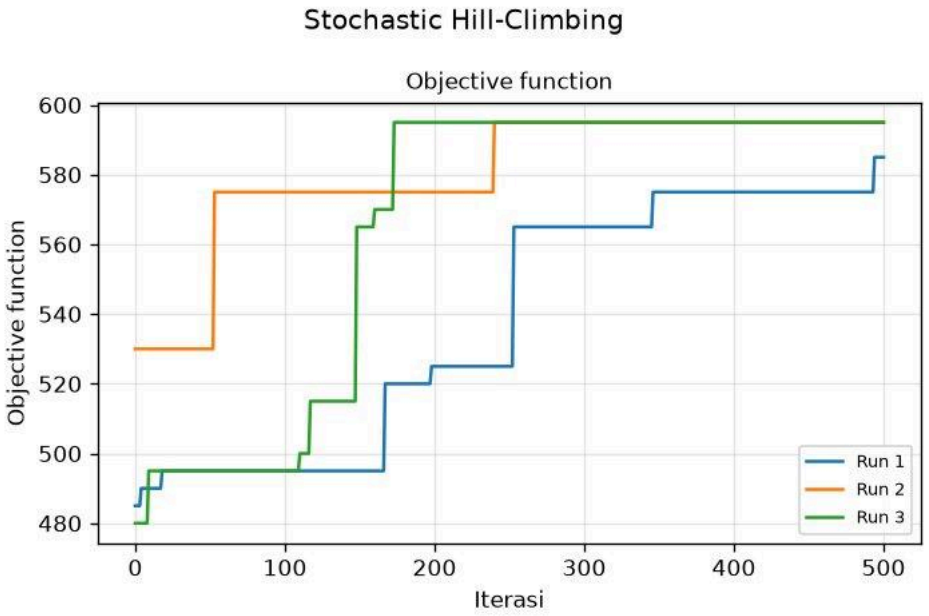
Berdasarkan **Gambar 2.2.2.1.1.**, algoritma memerlukan iterasi yang sangat banyak untuk mendapatkan *neighbor state* dengan fungsi objektif lebih baik. Solusi tidak mencapai global optima dan cenderung terjebak karena pemilihan neighbor tiap iterasi bersifat acak. Dengan semakin banyaknya opsi state yang bisa berubah, peluang menemukan *neighbor state* yang tepat juga semakin rendah.

Tabel 2.2.2.1.1 *Value* awal, *value* akhir, jumlah iterasi, dan durasi untuk *Stochastic Hill Climbing*.

Run ke-	Value Awal	Value Akhir	Iterasi	Durasi
1	485	585	500	0.03 s
2	530	595	500	0.02 s
3	480	595	500	0.03 s

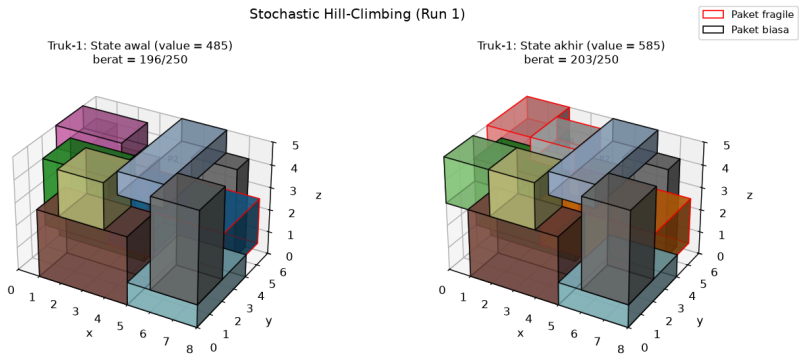
Terlihat pada **Tabel 2.2.2.1.1.** Bahwa meskipun durasi running algoritma cepat (karena tidak ada sorting per iterasi yang menyebabkan $O(n)$). Hasil yang didapat oleh algoritma Steepest Ascent Hill Climbing. Hal ini karena kekurangan dari algoritma ini adalah kebalikannya dari Steepest Ascent, jika pada Steepest Ascent yang kurang adalah proses diversifikasi, pada algoritma Stochastic Hill Climbing

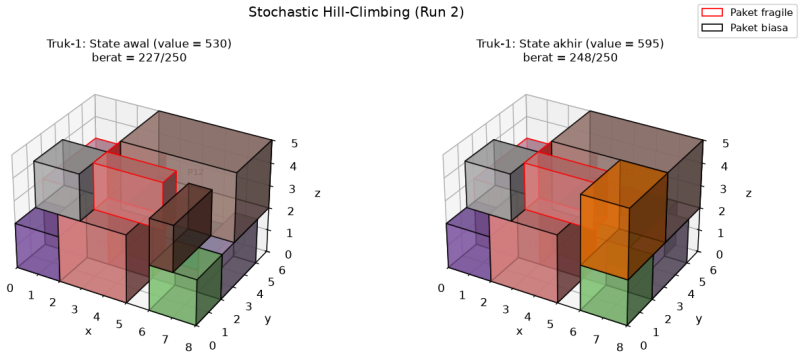
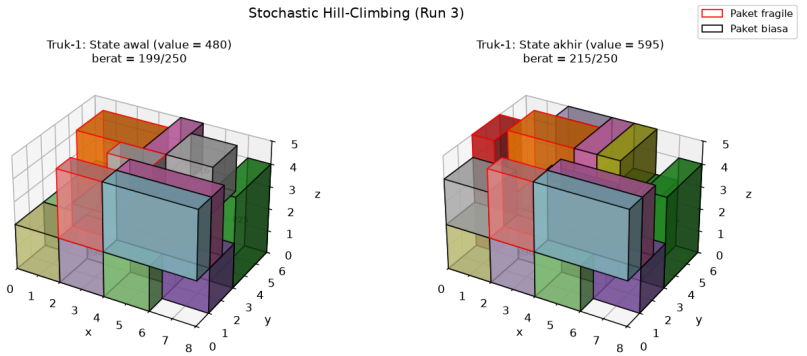
justru yang kurang adalah aspek intensifikasi/eksploitasi dari solusi. Karena pengambilan neighbor yang sangat random menyebabkan arah kenaikan objektif yang tidak jelas, sehingga menyebabkan algoritma membutuhkan iterasi yang sangat banyak untuk dapat mencapai state optimal, terlihat dari running bahwa meskipun dengan batas iterasi maksimum 500 algoritma masih belum dapat mencapai global optimanya dan hasilnya masih cenderung kurang dibandingkan oleh Steepest Ascent



Gambar 2.2.1.2.1 Objektif Terhadap Iterasi pada *Stochastic Hill-Climbing*.

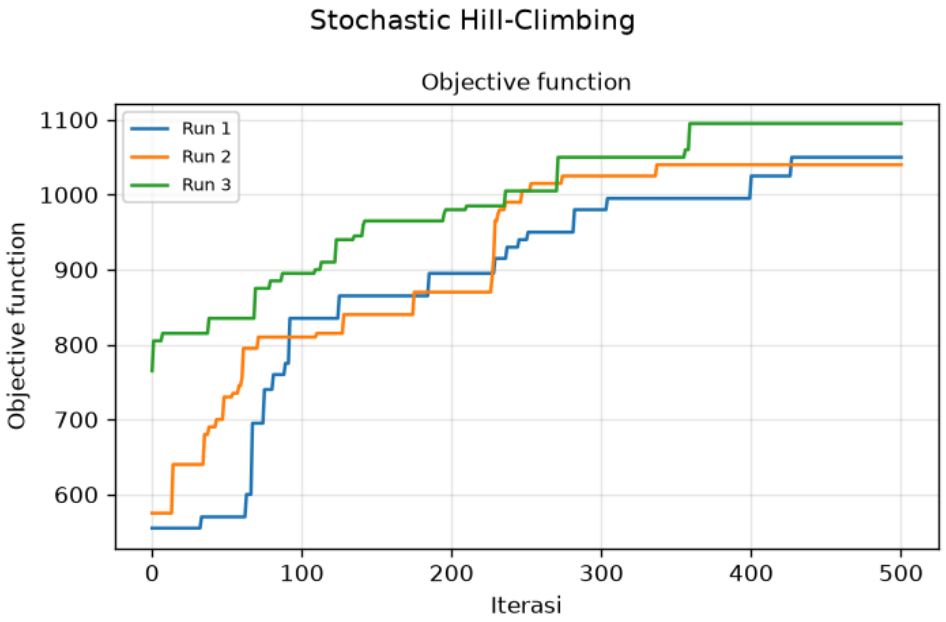
Tabel 2.2.2.1.2 Visualisasi penempatan paket pada truk pada iterasi awal dan akhir algoritma *Stochastic Hill-Climbing*.

Run ke-	Visualisasi Penempatan Paket pada Truk
1	

Run ke-	Visualisasi Penempatan Paket pada Truk
2	
3	

2.2.2.2. Multi Truk

Algoritma kemudian dicoba untuk menyelesaikan persoalan multi-truk hingga 500 iterasi dengan spesifikasi paket yang berbeda dari spesifikasi paket untuk satu. Algoritma mampu secara konsisten menghasilkan solusi dengan fungsi objektif diatas 1000. Hal ini baik karena algoritma mampu menghasilkan solusi secara konsisten dengan fungsi objektif tinggi. Namun, perlu diperhatikan bahwa jumlah iterasi yang banyak diperlukan untuk mendapatkan solusi yang baik. Algoritma ini cocok apabila diperlukan solusi yang konsisten menghasilkan objective function tinggi namun waktu (banyaknya iterasi tidak menjadi constraint). Terlihat juga bahwa semakin tinggi fungsi objektif, algoritma akan semakin memerlukan jumlah iterasi yang banyak akibat “jarangnya” ada solusi yang lebih baik daripada *current state*.

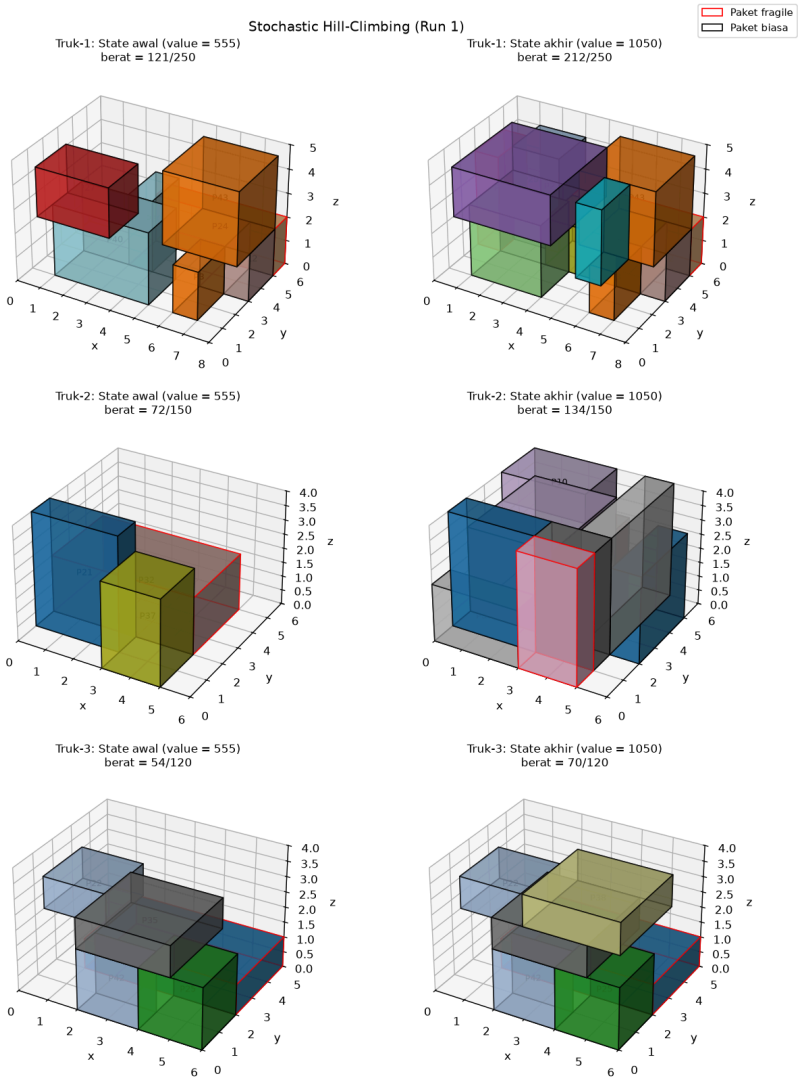


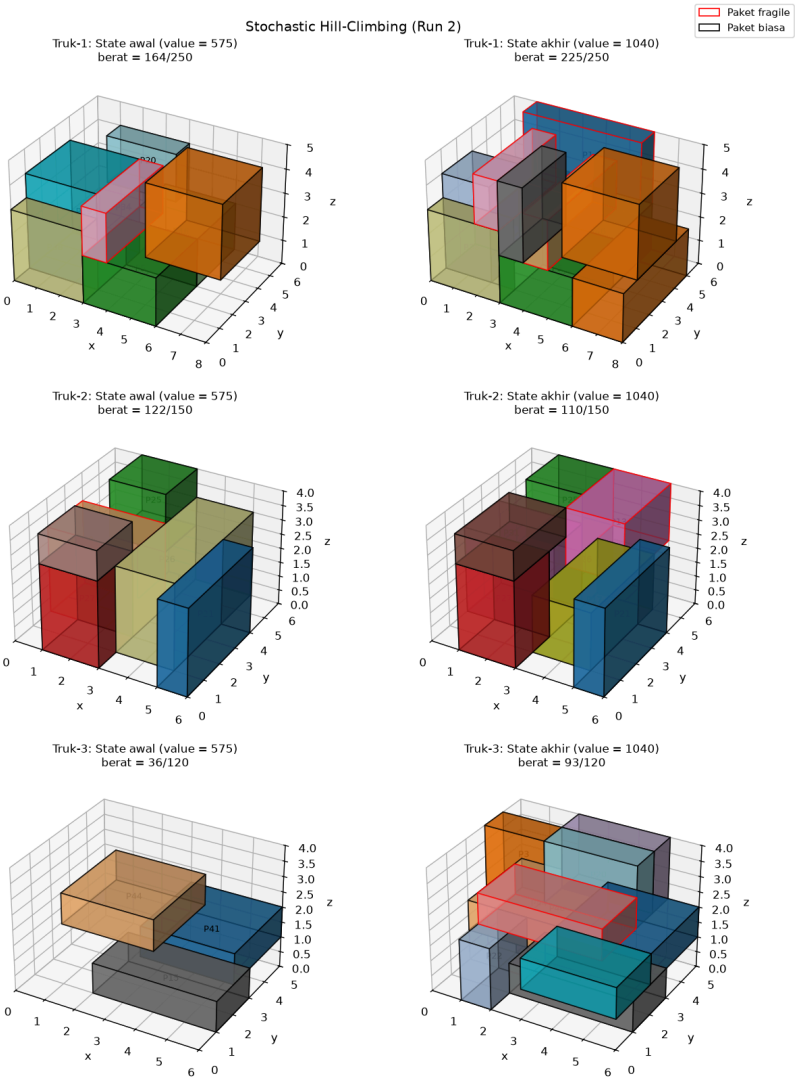
Gambar 2.2.2.2.1 Plot objektif terhadap iterasi pada *Stochastic Hill-Climbing* Multi Truck.

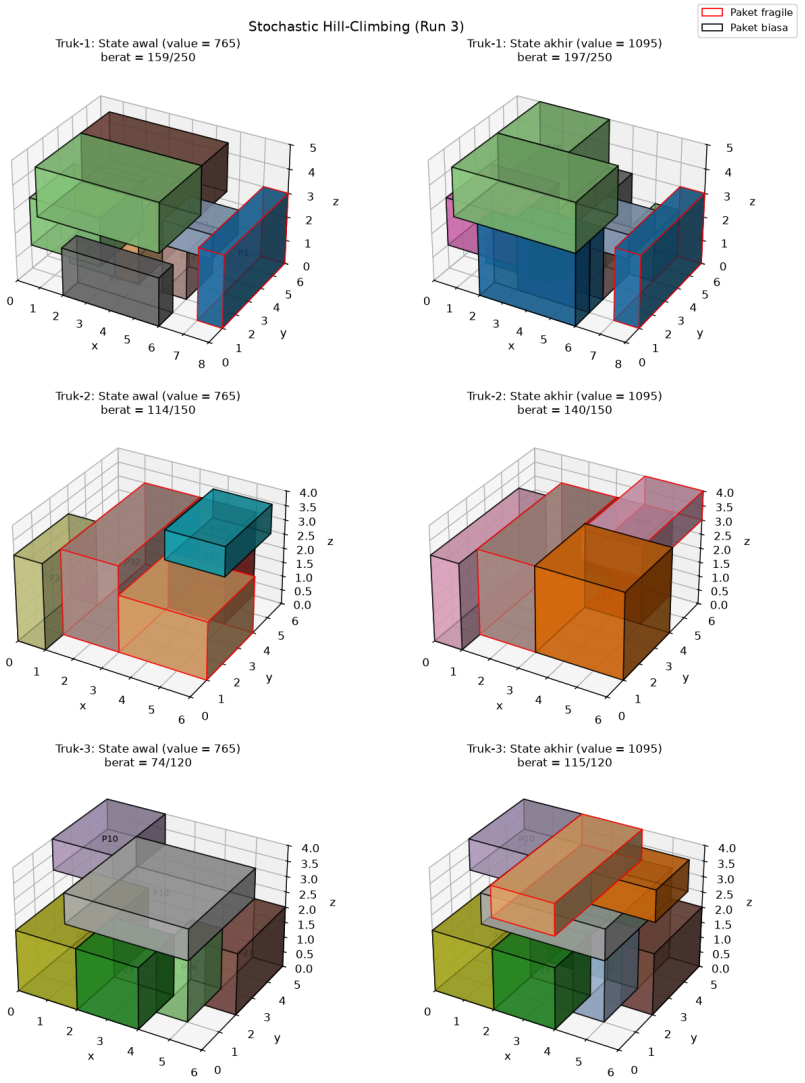
Tabel 2.2.2.1.1 Value awal, value akhir, jumlah iterasi, dan durasi untuk Stochastic Hill Climbing Multi-Truck.

Run ke-	Value Awal	Value Akhir	Iterasi	Durasi
1	690	1110	500	0.02 s
2	625	970	500	0.02 s
3	715	1100	500	0.03 s

Tabel 2.2.2.2.2 Visualisasi penempatan paket pada truk pada iterasi awal dan akhir algoritma Stochastic Hill-Climbing Multi Truck.

Run ke-	Visualisasi Penempatan Paket pada Multi Truk
1	

Run ke-	Visualisasi Penempatan Paket pada Multi Truk
2	

Run ke-	Visualisasi Penempatan Paket pada Multi Truk
3	

2.2.3. Random Restart Hill Climbing

Strategi *random restart* diterapkan untuk mengatasi terjebak dalam solusi lokal optima. Algoritma berjalan dengan melakukan proses *hill climbing* dari beberapa *initial state* acak secara berulang. Algoritma akan berhenti ketika jumlah restart telah mencapai batas *max_restarts* (dalam kasus ini adalah 5x *restart*). Solusi terbaik secara global dari sekian banyak *run/restart* dipilih sebagai hasil akhir. Hal ini bertujuan untuk melakukan pencarian lebih luas dan memperbesar peluang mendekati solusi global optima.

Kode 2.2.2.1. Kode Implementasi dari *Random Restart Hill-Climbing*

```
class RandomRestartHC(HillClimbing):
    name = "Random Restart Hill-Climbing"
```

```
    iteration_label = "Iterasi (seluruh restart disambung)"

    def __init__(self, problem: Problem, max_restarts: int) -> None:
        super().__init__(problem)
        self.max_restarts = max_restarts

    def parameter(self) -> dict[str, Any]:
        return {"max_restarts": self.max_restarts}

    def search(self) -> SearchResult:
        # Run pertama + max_restarts kali restart, tiap run pake Steepest Ascent
        # dari state awal random
        steepest = SteepestAscentHC(self.problem)
        restart_results = [steepest.search() for _ in range(self.max_restarts +
1)]
        best = max(restart_results,
                   key=lambda result: result.final_state.objective())

        # History semua restart disambung jadi satu kurva
        history = []
        for result in restart_results:
            history.extend(result.history)

        iterations_per_restart = [result.iterations for result in
                                  restart_results]
        return SearchResult(restart_results[0].initial_state, best.final_state,
                            history, iterations=sum(iterations_per_restart),
                            info={"restarts": self.max_restarts,
                                  "iterations_per_restart":
                                   iterations_per_restart})

    def metrics(self, result: SearchResult) -> dict[str, Any]:
        iterations_per_restart = result.info["iterations_per_restart"]
        return {"Restart": result.info["restarts"],
                "Iterasi per restart": ", ".join(str(iterations) for iterations
                                                 in iterations_per_restart)}
```

2.2.3.1. Satu Truk

Berdasarkan **Gambar 2.2.3.1.1**, algoritma mampu mendapatkan *state* dengan fungsi objektif lebih baik dibandingkan dengan state yang didapat dari algoritma *steepest ascent*. Algoritma *random restart* melakukan *steepest hill climbing* berkali-kali dalam sekali run tergantung banyaknya *max_restarts*. Namun dapat atau tidaknya solusi dengan fungsi objektif tinggi tidak dapat diprediksi karena tergantung initial state karena *state* diinisiasi secara acak.

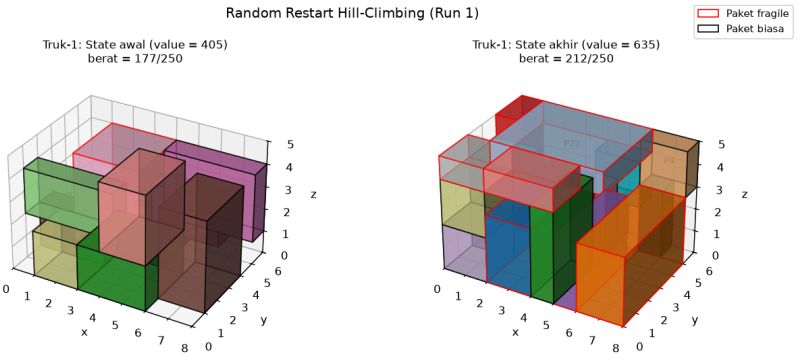
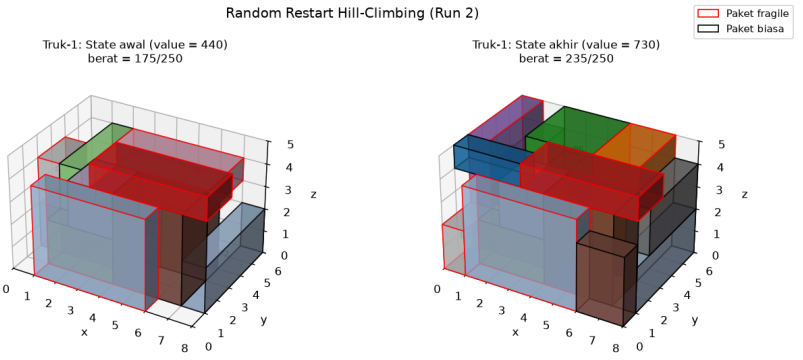
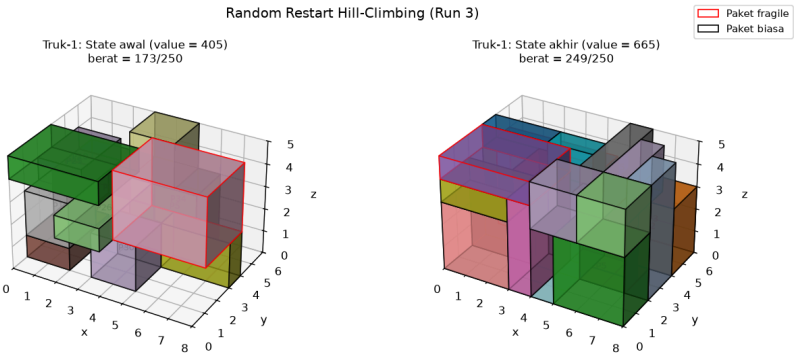


Gambar 2.2.3.1.1 Plot objektif terhadap iterasi pada *Random Restart Hill-Climbing*.

Tabel 2.2.2.1.1. Value awal, value akhir, jumlah iterasi, dan durasi untuk *Random Restart Hill Climbing*.

Run ke-	Value Awal	Value Akhir	Restart	Iterasi per restart	Durasi
1	450	675	5	7,6,10,8,10,9	0.73 s
2	605	675	5	4,8,7,8,10,8	0.69 s
3	470	680	5	6,8,9,9,3,8	0.65 s

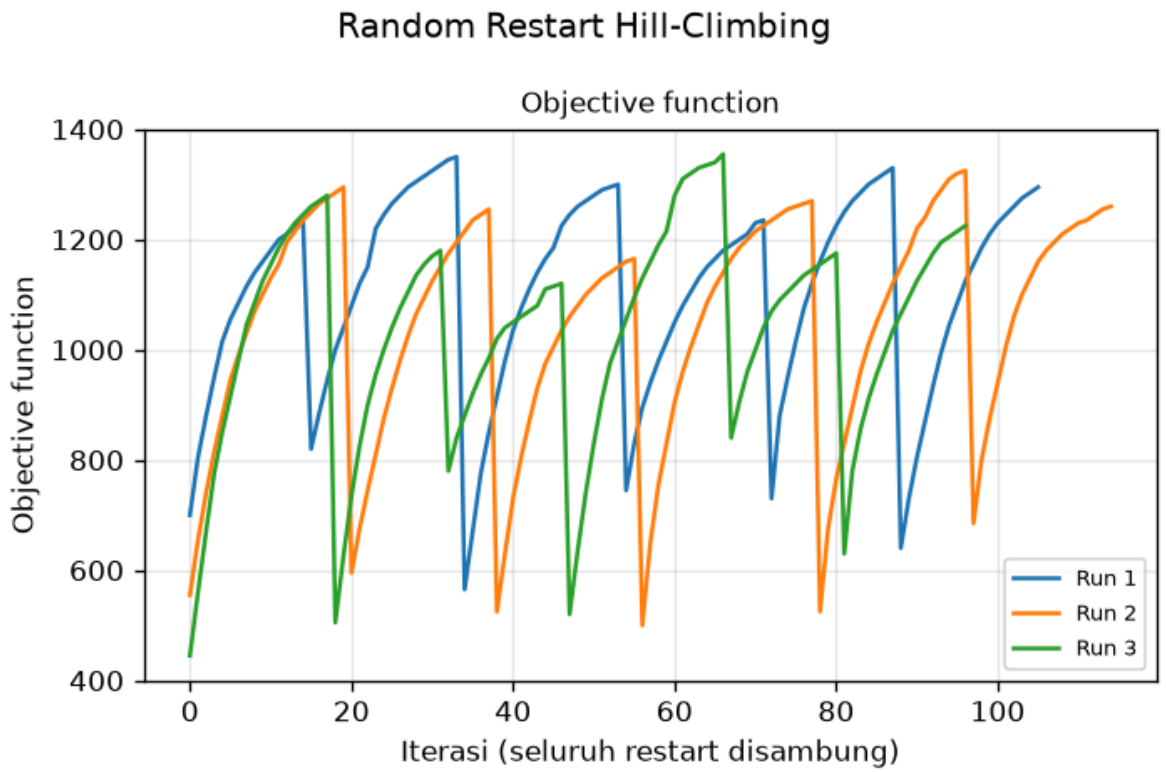
Tabel 2.2.2.1.2 Visualisasi penempatan paket pada truk pada iterasi awal dan akhir algoritma *Stochastic Hill-Climbing*.

Run ke-	Visualisasi penempatan Paket pada Truk
1	
2	
3	

2.2.3.2. Multi Truk

Algoritma kemudian dicoba untuk menyelesaikan persoalan multi-truk hingga mencapai *max_restarts* = *5*. Algoritma random restart terlihat secara konsisten menghasilkan solusi dengan fungsi objektif diatas 1000 bahkan diatas 1200. Karena dalam satu kali menjalankan program, akan ada sebanyak *max_restarts* initial state yang akan dieksplorasi. Model kemudian memilih solusi yang terbaik dari solusi *hill climbing* tiap *restart* sebagai solusi. Algoritma mampu menghasilkan solusi yang sangat baik apabila *max_restarts* cukup banyak dan penemuan solusi global optima bersifat random sehingga pada suatu *run* dengan *run* lainnya bisa tidak ditemukan solusi optima yang sama. Algoritma baik

digunakan apabila memiliki kemampuan komputasi tinggi karena untuk mendapatkan solusi yang betul-betul optima, diperlukan max_restarts yang banyak pada satu kali *run*. Hal tersebut tentunya memerlukan waktu yang lama, namun dengan kompensasi penemuan solusi yang relatif lebih baik.

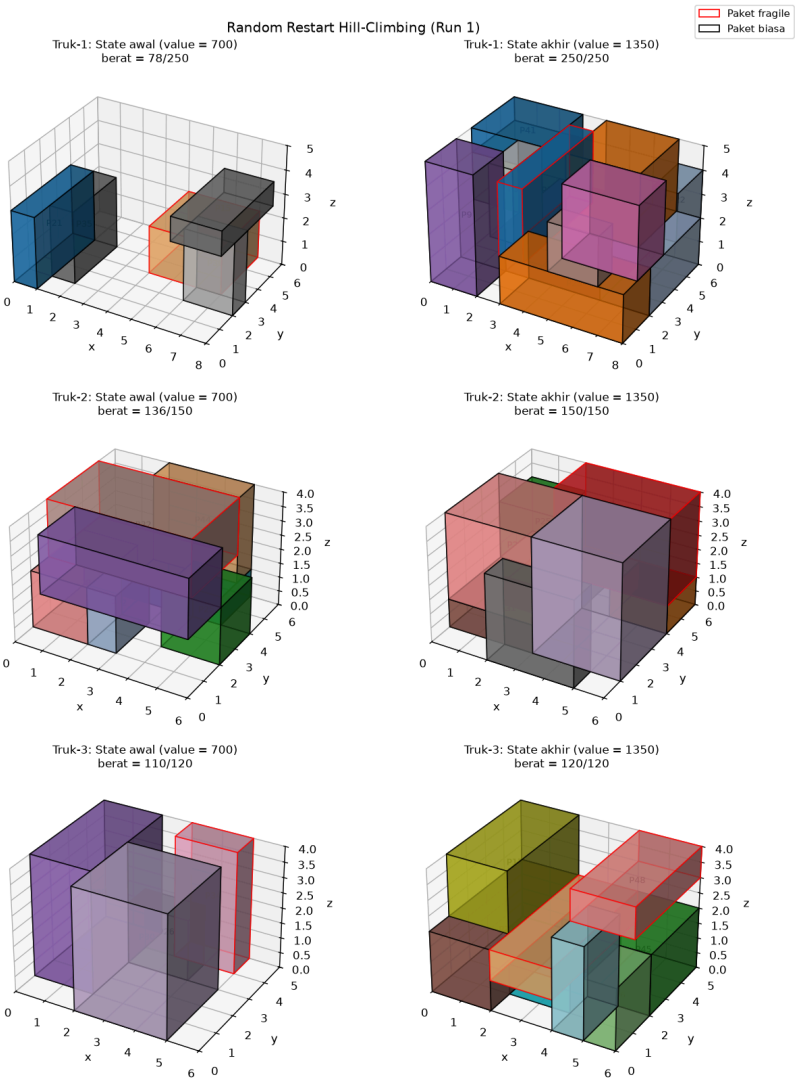


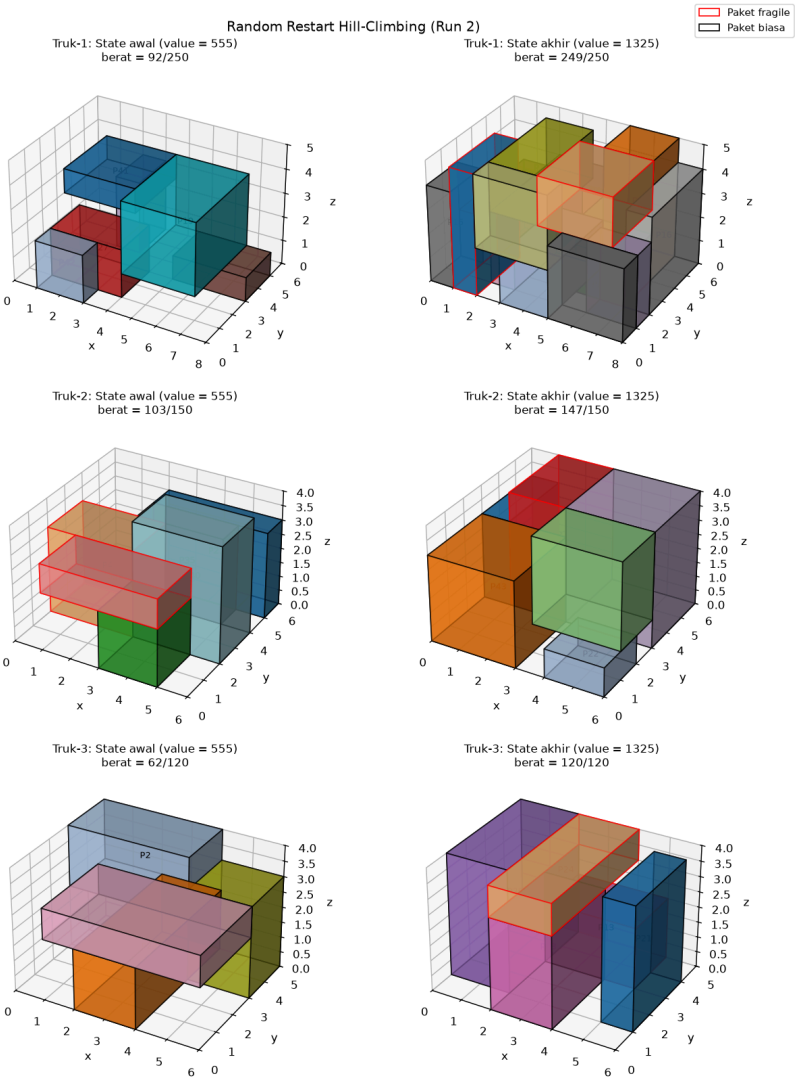
Gambar 2.2.3.1.1 Plot objektif terhadap iterasi pada *Random Restart Hill-Climbing Multi Truck*.

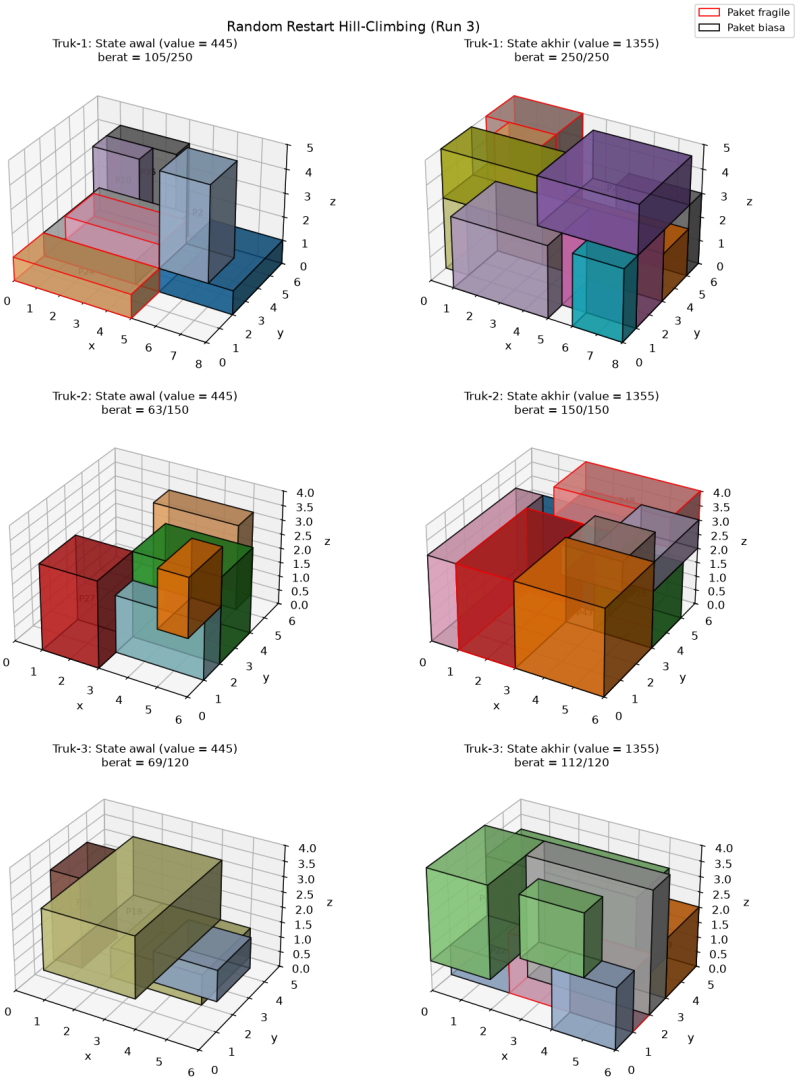
Tabel 2.2.2.1.1. Value awal, value akhir, jumlah iterasi, dan durasi untuk Random Restart Hill Climbing Multi Truck.

Run ke-	Value Awal	Value Akhir	Restart	Durasi
1	690	1350	5	2.58 s
2	625	1360	5	2.54 s
3	715	1400	5	2.58 s

Tabel 2.2.2.1.2 Visualisasi penempatan paket pada truk pada iterasi awal dan akhir algoritma Random Restart Hill-Climbing Multi Truck.

Run ke-	Visualisasi Penempatan Paket pada Multi Truk
1	

Run ke-	Visualisasi Penempatan Paket pada Multi Truk
2	

Run ke-	Visualisasi Penempatan Paket pada Multi Truk
3	

2.2.4. Sideways Move Hill Climbing

Algoritma ini mengatasi masalah dataran (*plateau*), nilai fungsi objektif antar tetangga cenderung tidak lebih besar, varian ini memperbolehkan algoritma bergerak ke tetangga yang memiliki nilai objektif setara (*sideways move*). Pergerakan dibatasi oleh parameter *maks_sideways* untuk menghindari terjebak dalam *loop* di area datar terus menerus.

Kode 2.2.2.1. Kode implementasi dari *Sideways Move Hill-Climbing.*

```
class SidewaysMoveHC(HillClimbing):
    name = "Hill-Climbing with Sideways Move"

    def __init__(self, problem: Problem, max_sideways: int) -> None:
```

```
        super().__init__(problem)
        self.max_sideways = max_sideways

    def parameter(self) -> dict[str, Any]:
        return {"max_sideways": self.max_sideways}

    def search(self) -> SearchResult:
        initial_state = State.random_initial(self.problem)
        state = initial_state
        history = [state.objective()]
        sideways_count = 0

        while True:
            neighbor_value, neighbor = self.best_successor(state)

            if neighbor_value < state.objective():
                break

            # Sideways move dibatasi max_sideways kali berturut-turut
            if neighbor_value == state.objective():
                if sideways_count >= self.max_sideways:
                    break
                sideways_count += 1
            else:
                sideways_count = 0

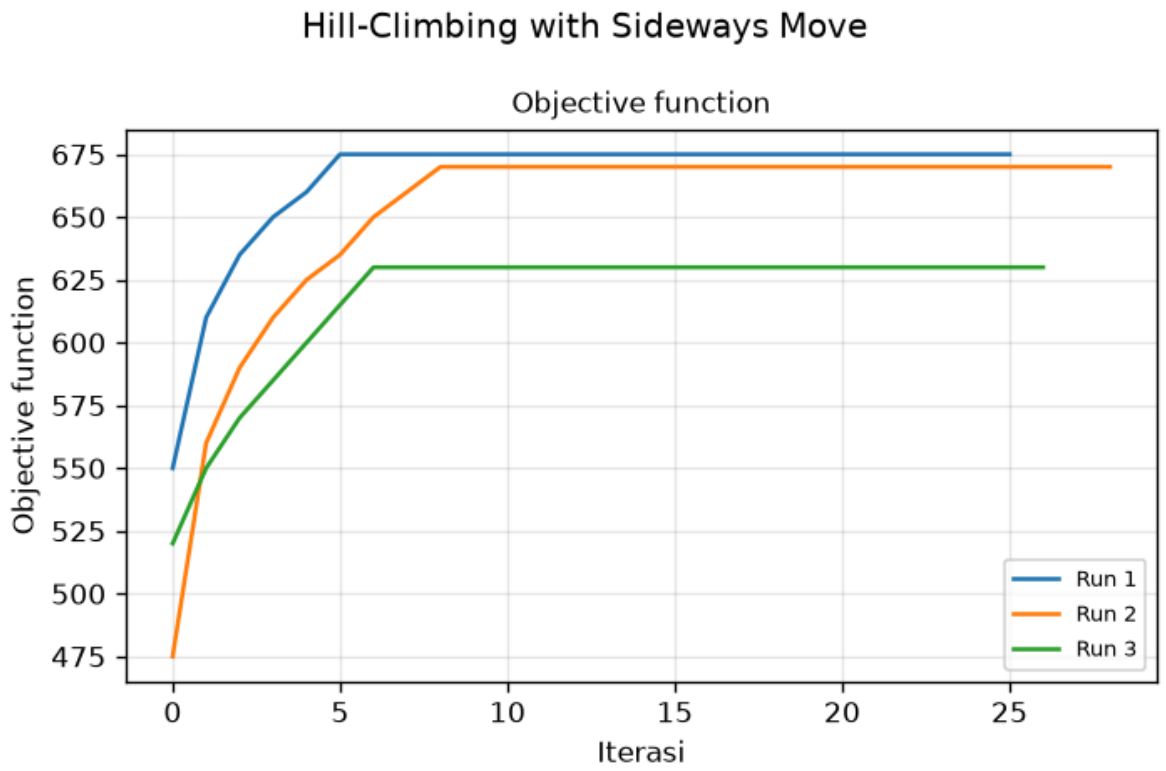
            state = neighbor
            history.append(neighbor_value)

        return SearchResult(initial_state, state, history,
iterations=len(history) - 1)
```

2.2.4.1. Satu Truk

Berdasarkan **Gambar 2.2.4.1.1** dan **Tabel 2.2.4.1.1**, ketiga *run* naik dengan cepat pada 5 sampai 8 iterasi pertama, sama seperti *steepest ascent*, lalu mendatar. Banyak iterasi setiap *run* selalu sama dengan banyak langkah naik ditambah 20 (25 = 5 + 20, 28 = 8 + 20, dan 26 = 6 + 20). Artinya, pada satu_truk tidak ada satu pun dari 20 *sideways move* yang membawa algoritma ke *state* yang lebih baik, sehingga pencarian berhenti karena batas *max_sideways* dan nilai akhir sama dengan nilai ketika *plateau* pertama kali dicapai.

Rerata nilai akhir (658,3) memang lebih tinggi daripada *steepest ascent* (618,3), tetapi karena *sideways move* tidak pernah menghasilkan kenaikan, selisih ini berasal dari state awal yang berbeda, bukan dari mekanisme *sideways move*. Pada persoalan ini, *plateau* umumnya terbentuk ketika truk hampir penuh (berat akhir 219/250 sampai 250/250). Langkah *move* dan *rotate* pada paket yang sudah berada di dalam truk tidak mengubah nilai, sehingga terdapat banyak *neighbor* bernilai sama, tetapi menambah paket baru sudah terhalang oleh kapasitas berat. Biaya durasinya pun kecil (0,19 sampai 0,33 detik).

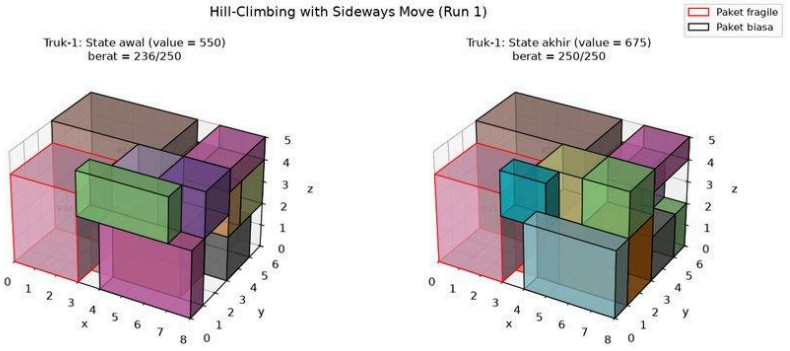
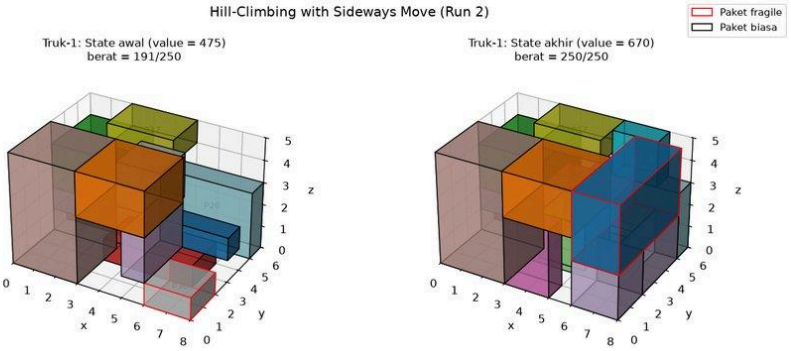
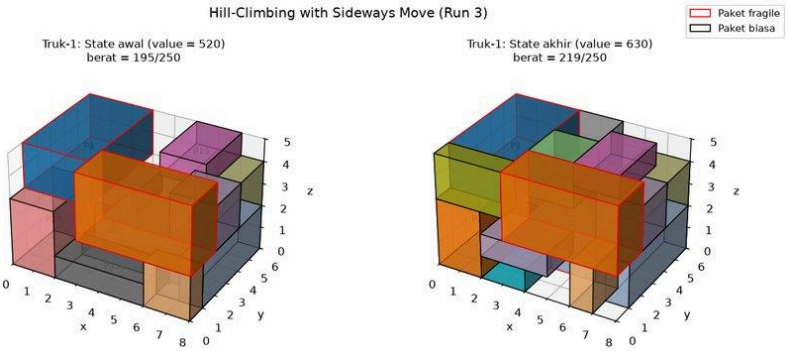


Gambar 2.2.4.1.1 Plot objektif terhadap iterasi pada *Sideways Move Hill-Climbing*.

Tabel 2.2.2.1.1. Value awal, value akhir, jumlah iterasi, dan durasi untuk Sideways Move Hill Climbing.

Run ke-	Value Awal	Value Akhir	Iterasi (naik + *sideways*)	Durasi
1	550	675	25 (5 + 20)	0.19 s
2	475	670	28 (8 + 20)	0.31 s
3	520	630	26 (6 + 20)	0.33 s

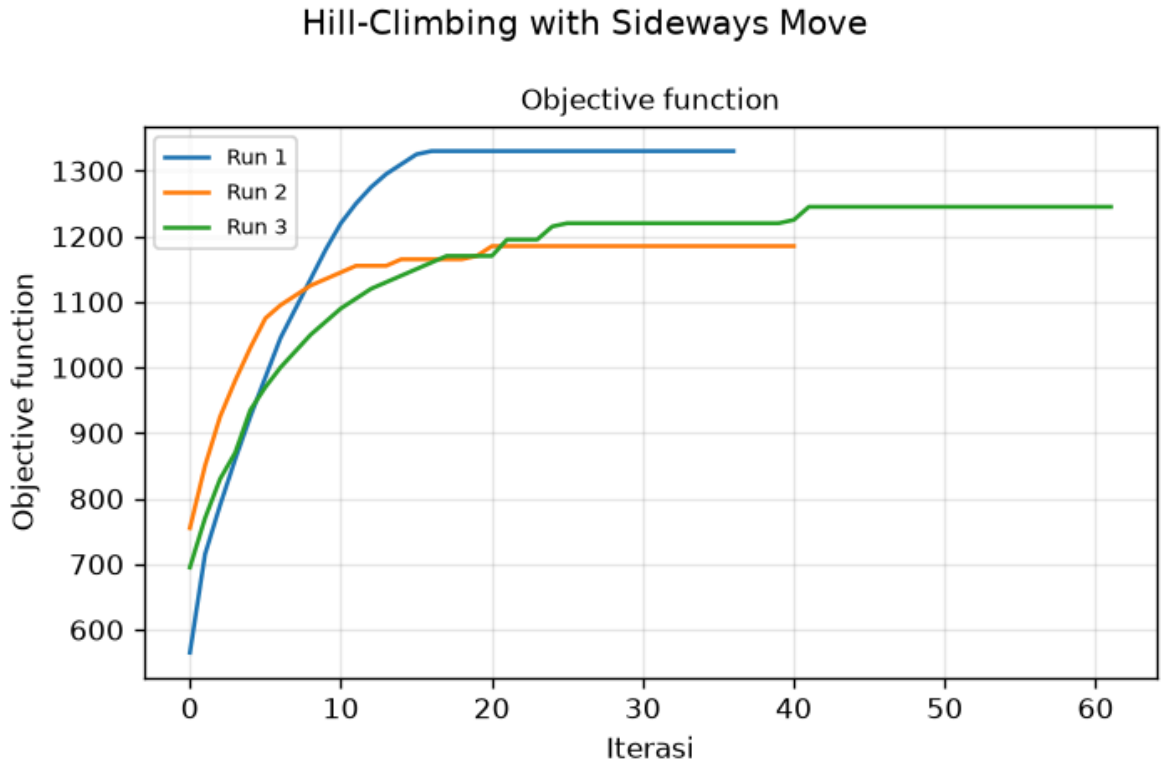
Tabel 2.2.2.1.2 Visualisasi penempatan paket pada truk pada iterasi awal dan akhir algoritma Sidewats Hill-Climbing.

Run ke-	Visualisasi Penempatan Paket pada Truk
1	
2	
3	

2.2.4.2. Multi Truk

Algoritma kemudian dicoba untuk menyelesaikan persoalan multi-truk dan apabila memilih jalur datar sebanyak 20 kali berturut-turut (*max_sideways* = 20) maka program akan berhenti. Algoritma dapat menghasilkan solusi dengan fungsi objektif secara konsisten di atas 1.100 (1.185 sampai 1.330). Berbeda dengan satu_truk, pada multi_truk *sideways move* terbukti berguna pada dua dari tiga *run*. Pada *run* 2, setelah berjalan mendatar di nilai 1.155, algoritma menemukan tiga kenaikan lagi hingga 1.185. Pada *run* 3, algoritma keluar dari *plateau* di nilai 1.170 dan naik lima kali hingga 1.245, termasuk dua kenaikan pada iterasi ke-40 dan ke-41 setelah *plateau* panjang (terlihat pada Gambar 2.2.4.2.1). Hal ini terjadi karena tiga truk memberi lebih banyak ruang untuk menggeser paket sehingga ruang kosong yang cukup untuk paket baru dapat terbentuk. Dengan iterasi di bawah 65, algoritma mampu menghasilkan solusi dengan fungsi

objektif cukup tinggi, meskipun masih terjebak dalam lokal optima. Algoritma baik digunakan pada persoalan time constraint menengah dan memerlukan solusi yang lebih konsisten menghasilkan solusi dengan fungsi cukup objektif tinggi.

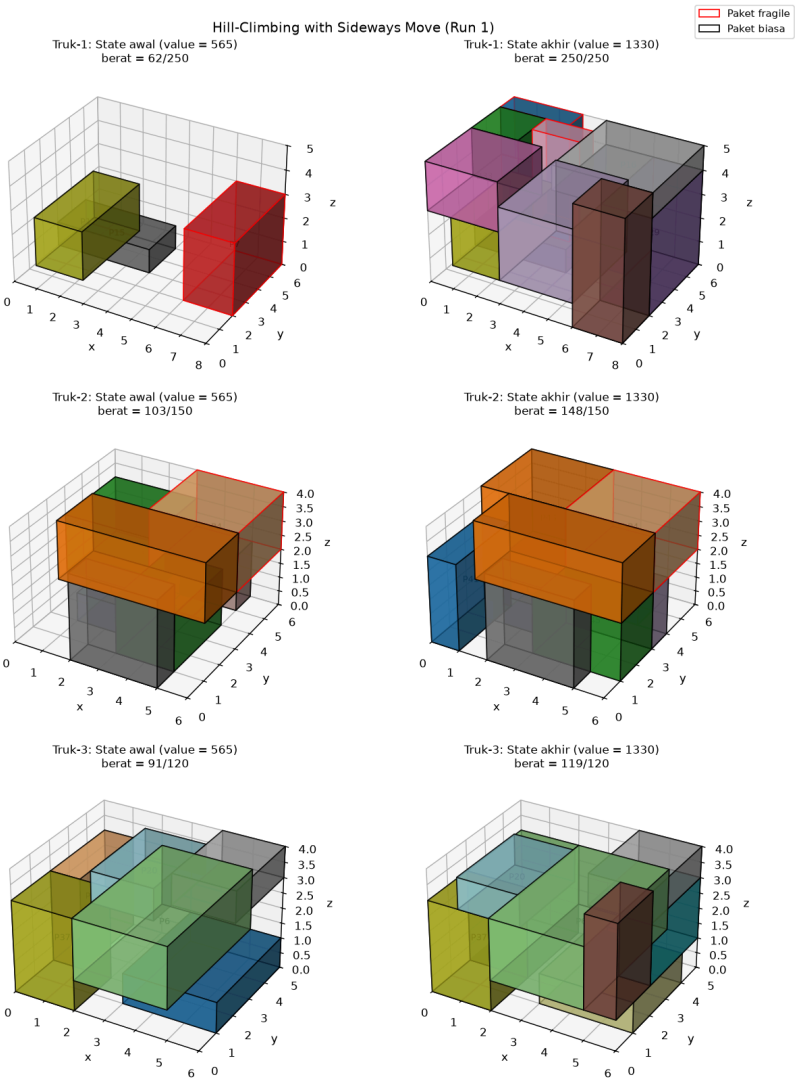


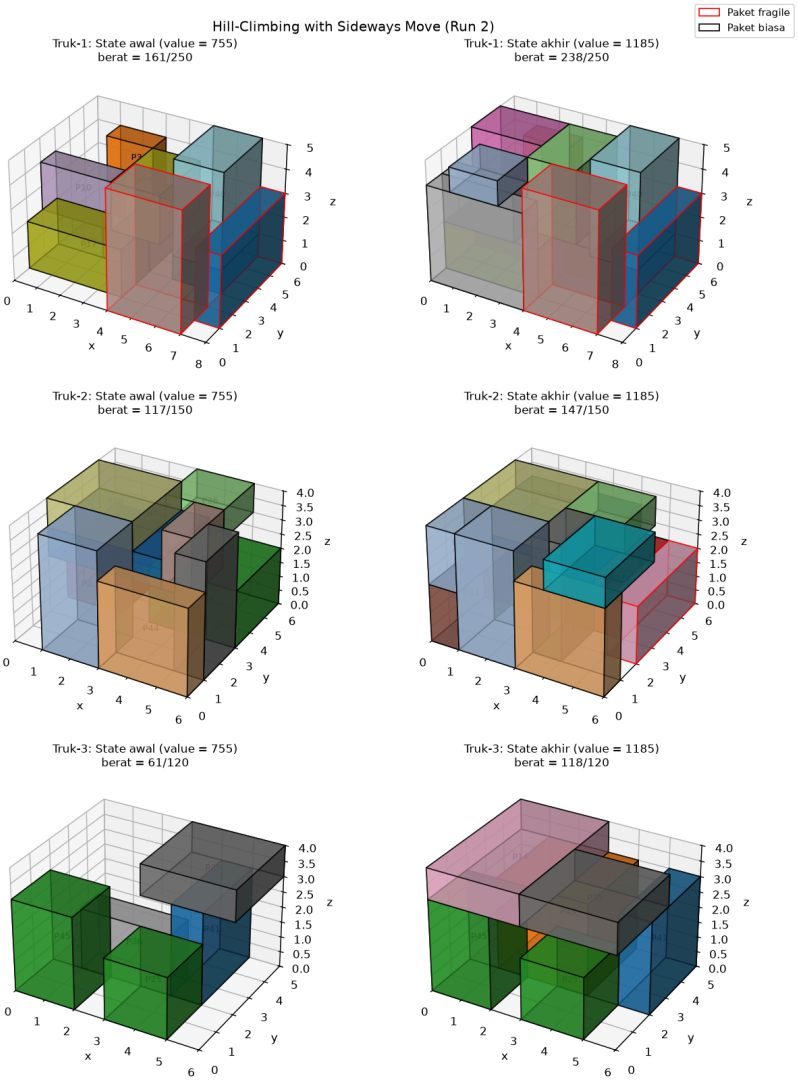
Gambar 2.2.4.1.1 Plot objektif terhadap iterasi pada *Sideways Move Hill-Climbing Multi Truck*.

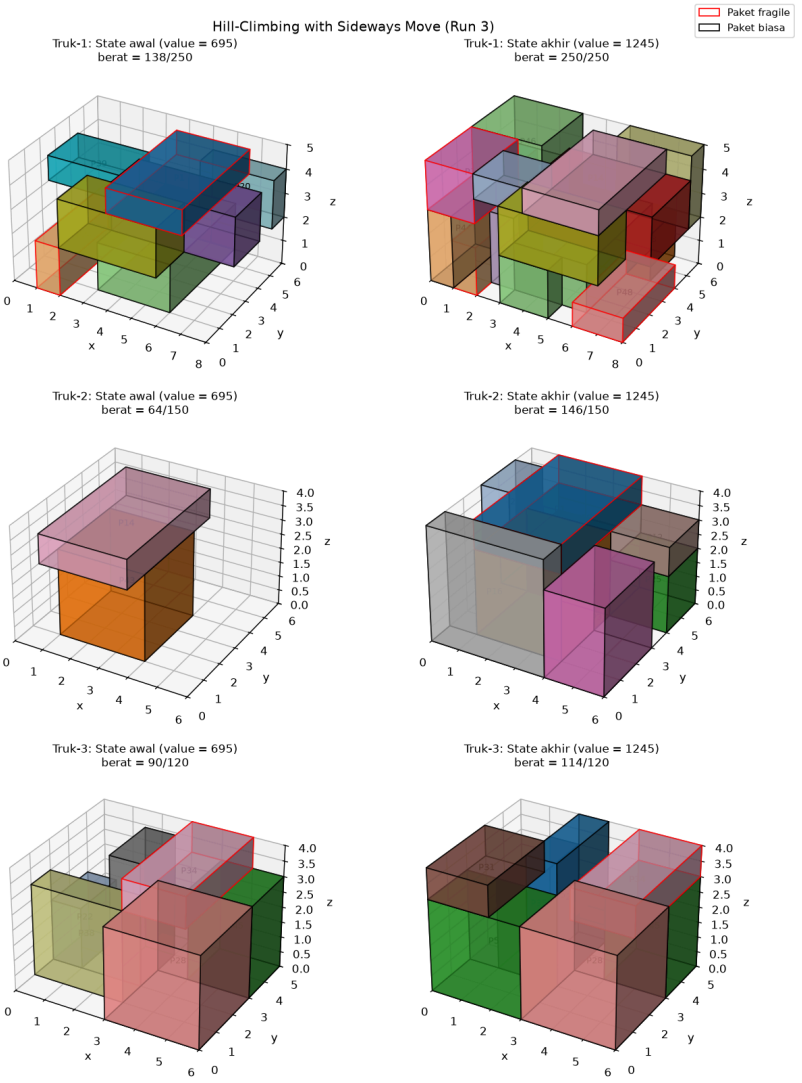
Tabel 2.2.2.1.1. Value awal, value akhir, jumlah iterasi, dan durasi untuk Sideways Move Hill Climbing Multi Truck.

Run ke-	Value Awal	Value Akhir	Iterasi (naik + *sideways*)	Durasi
1	565	1330	36 (16 + 20)	0.93 s
2	755	1185	40 (14 + 26)	1.22 s
3	695	1245	61 (22 + 39)	1.56 s

Tabel 2.2.2.1.2 Visualisasi penempatan paket pada truk pada iterasi awal dan akhir algoritma Sideways Hill-Climbing

Run ke-	Visualisasi Penempatan Paket pada Truk
1	

Run ke-	Visualisasi Penempatan Paket pada Truk
2	

Run ke-	Visualisasi Penempatan Paket pada Truk
3	

2.3. Implementasi *Simulated Annealing*

Simulated Annealing yang diimplementasikan mengikuti algoritma pada Russell (2020). Pada tiap iterasi dibangkitkan satu *successor* acak. Dengan ΔE = nilai(n) - nilai(c), *neighbor* yang lebih baik ($\Delta E > 0$) selalu diterima, sedangkan *neighbor* yang tidak lebih baik diterima dengan peluang $P = e^{\Delta E/T}$ dibandingkan dengan bilangan acak seragam pada [0, 1). Pada T tinggi, SA mendekati *random walk*, dan pada T mendekati 0, SA mendekati *stochastic hill climbing*.

Temperatur turun secara geometrik, $T_{t+1} = \alpha T_t$ dengan T_0 dan α sebagai parameter. Pencarian berhenti ketika $T \leq T_{min}$ (pengganti syarat $T = 0$ yang tidak dapat tercapai pada jadwal geometrik),

sehingga banyaknya iterasi, yaitu $\lceil \ln(T_{min}/T_0)/\ln \alpha \rceil$, ditentukan sepenuhnya oleh ketiga parameter ini. Pada tiap iterasi dengan $\Delta E < 0$, nilai $e^{\Delta E/T}$ dicatat untuk diplot terhadap iterasi, sedangkan iterasi dengan $\Delta E \geq 0$ tidak dicatat pada plot peluang.

Selain itu, dihitung juga frekuensi *stuck*. Pencarian dianggap *stuck* di *local optima* apabila nilai objektif tidak berubah selama `stuck_limit` iterasi berturut-turut.

Kode 2.3.1. Pencarian *Simulated Annealing*.

```
def search(self) -> SearchResult:
    # Temperatur turun geometrik
    initial_state = State.random_initial(self.problem)
    state = initial_state
    T = self.T0
    history = [state.objective()]
    history_prob: list[float] = []   # e^(delta_E/T) or NaN
    stuck_count = 0
    unchanged_iterations = 0

    while T > self.T_min:
        neighbor = state.random_successor()
        delta_E = neighbor.objective() - state.objective()

        if delta_E > 0:
            state = neighbor
            history_prob.append(float("nan"))
        else:
            prob = math.exp(delta_E / T)
            if delta_E < 0:
                history_prob.append(prob)
            else:
                history_prob.append(float("nan"))
            if random.random() < prob:
                state = neighbor

        # Dihitung stuck jika value tidak berubah selama stuck_limit iterasi
        # berturut-turut
        if state.objective() == history[-1]:
            unchanged_iterations += 1
        else:
            unchanged_iterations = 0
        if unchanged_iterations == self.stuck_limit:
```

```
            stuck_count += 1
            unchanged_iterations = 0

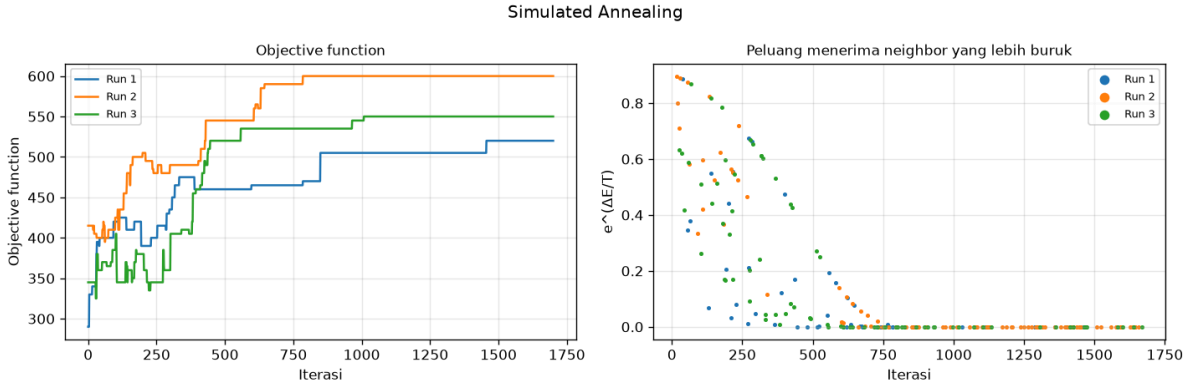
        history.append(state.objective())
        T = self.alpha * T

    return SearchResult(initial_state, state, history,
                        iterations=len(history) - 1,
                        info={"stuck": stuck_count,
                        "history_prob": history_prob})
```

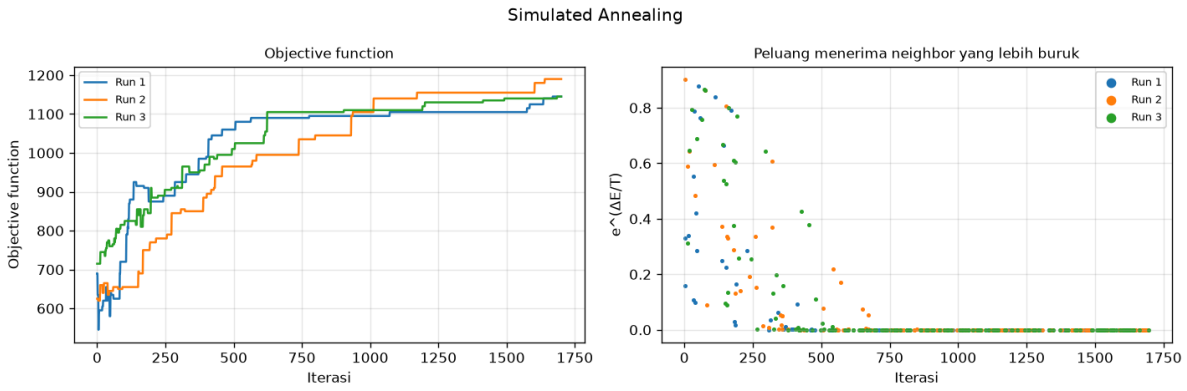
Tabel 2.3.1 memuat hasil SA pada kedua dataset, dan **Gambar 2.3.1** serta **Gambar 2.3.2** memuat kurva nilai objektif (kiri) dan plot peluang penerimaan $e^{\Delta E/T}$ terhadap iterasi (kanan).

Tabel 2.3.1. Hasil SA pada `satu_truk` dan `multi_truk`.

Run	*Objective Function*			Iterasi	Frekuensi *Stuck*	Durasi (s)
	Awal	Akhir	Terbaik			
`satu_truk`						
1	290	520	520	1700	11	0,10
2	415	600	600	1700	12	0,09
3	345	550	550	1700	11	0,10
`multi_truk`						
1	690	1145	1145	1700	9	0,15
2	625	1190	1190	1700	8	0,16
3	715	1145	1145	1700	8	0,15

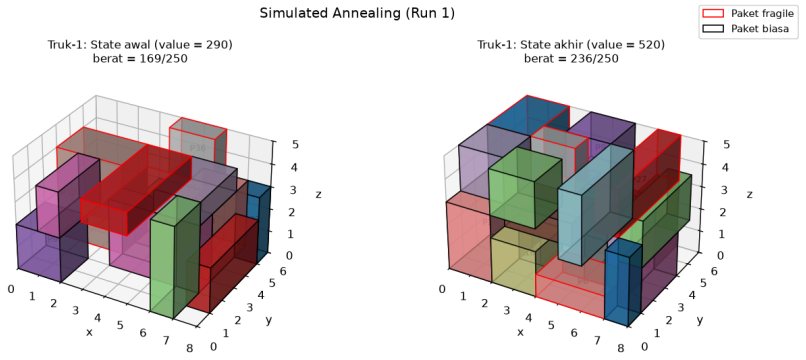
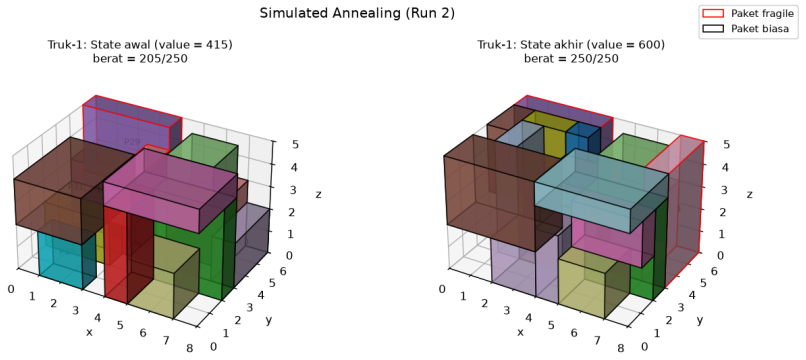
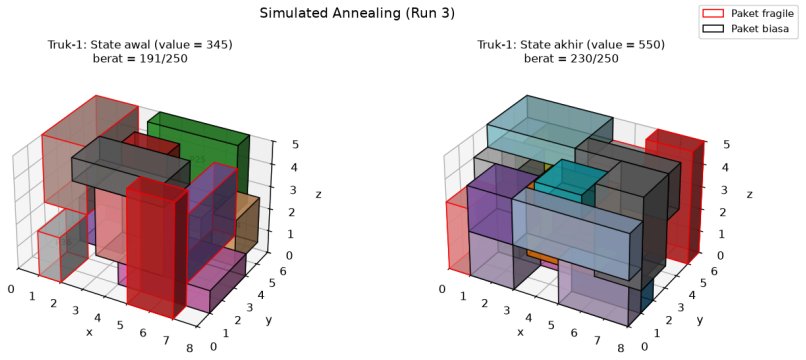


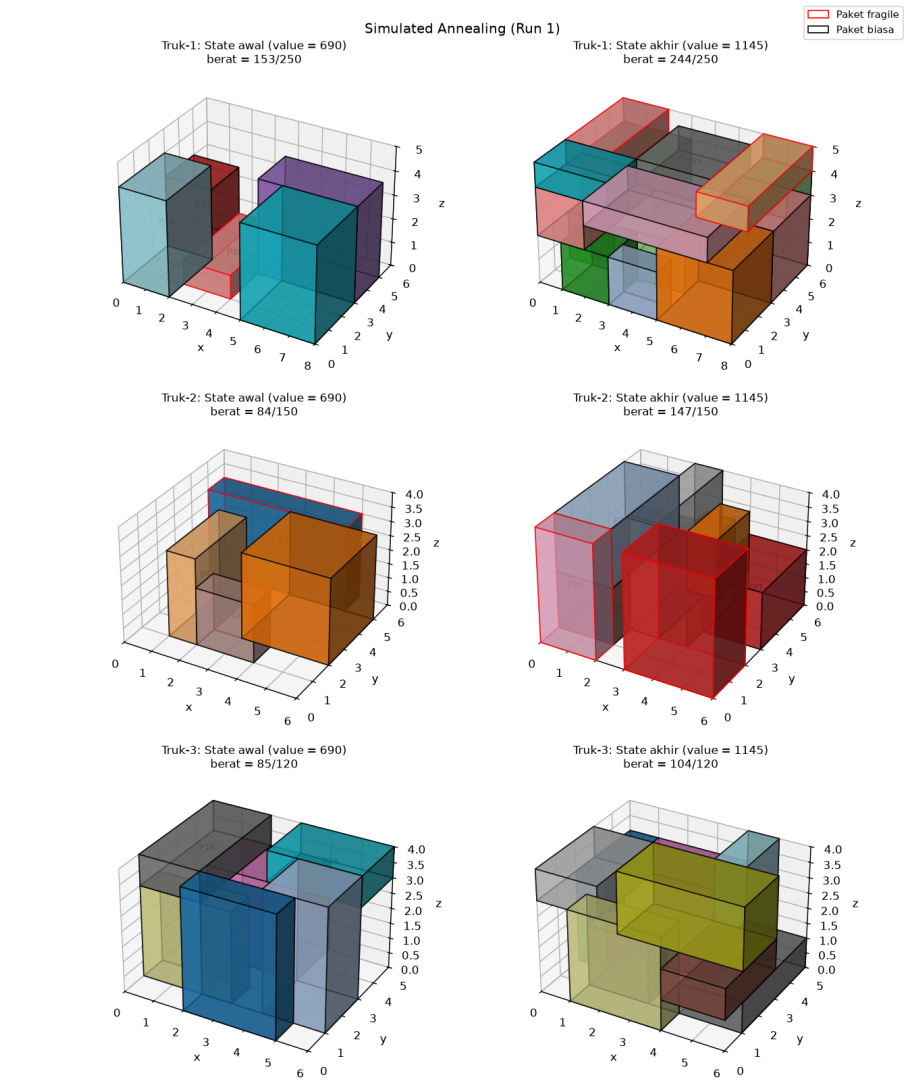
Gambar 2.3.1. SA pada `satu_truk`.

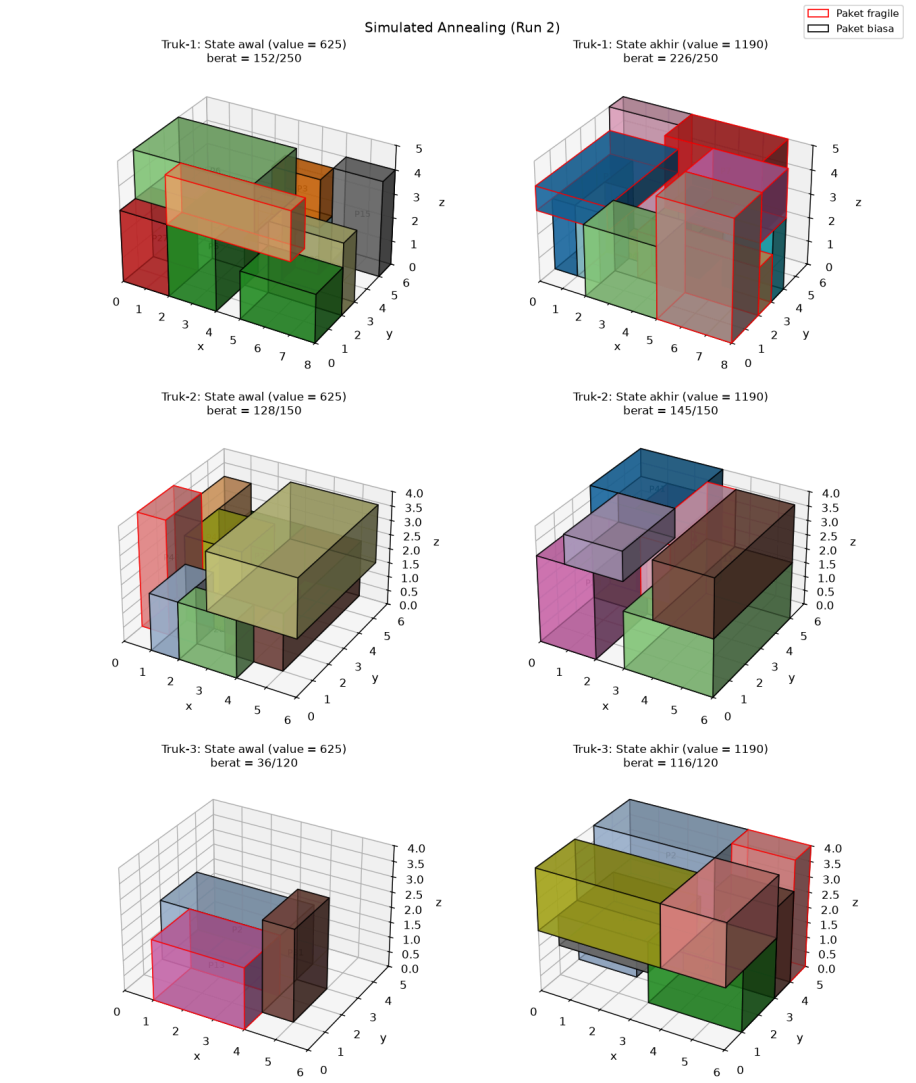


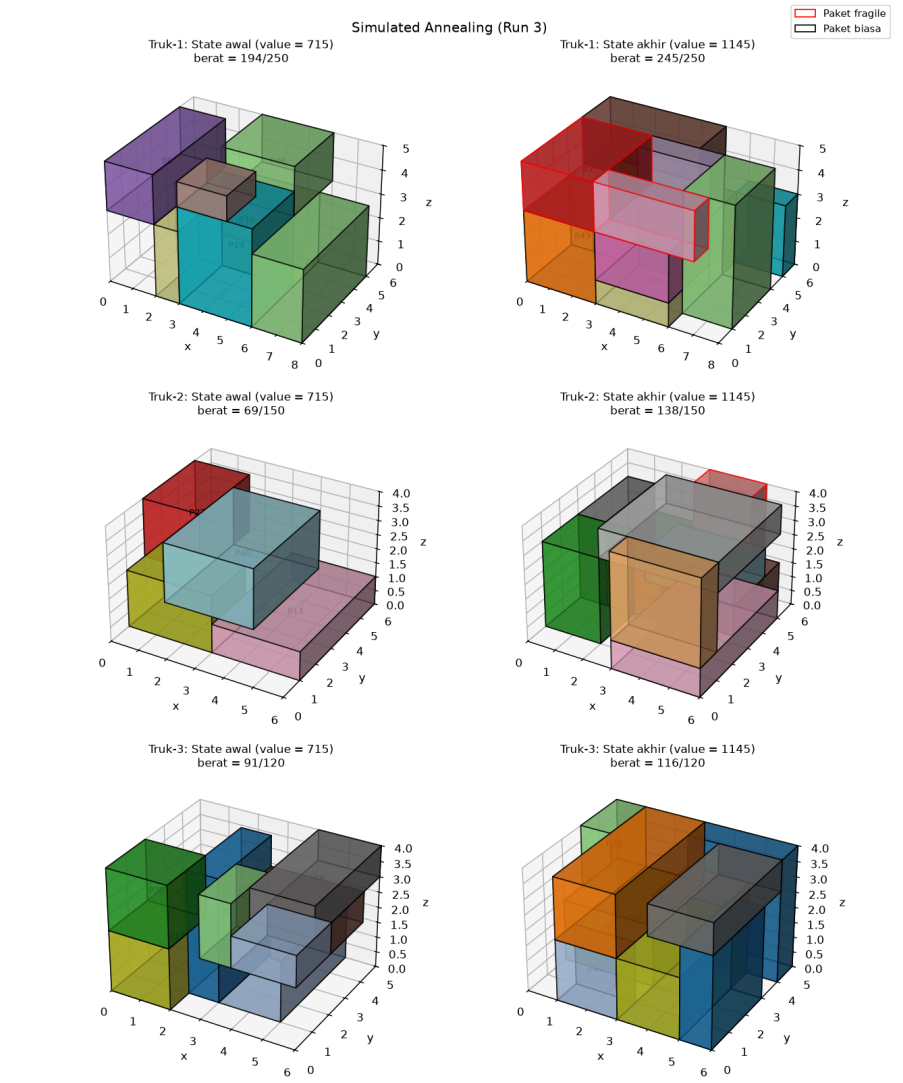
Gambar 2.3.2. SA pada `multi_truk`.

Tabel 2.3.2. Visualisasi hasil akhir SA pada `satu_truk` dan `multi_truk`.

***Run* ke-**	**Visualisasi penempatan Paket pada Truk**
`satu_truk`	
1	
2	
3	

Run ke-	Visualisasi penempatan Paket pada Truk
`multi_truk`	
1	

Run ke-	Visualisasi penempatan Paket pada Truk
`multi_truk`	
2	

Run ke-	Visualisasi penempatan Paket pada Truk
`multi_truk`	
3	

Kurva nilai objektif memperlihatkan bahwa pada awal pencarian (T tinggi) nilai berfluktuasi dan sering turun, karena *neighbor* yang lebih buruk diterima dengan peluang yang besar. Rerata peluang penerimaan pada 200 iterasi pertama *run* 1 adalah 0,41 pada `satu_truk` dan 0,56 pada `multi_truk`. Ketika T menurun, kurva naik secara umum, lalu mendatar. Plot peluang pada panel kanan menunjukkan bahwa pada *run* 1 peluang menerima *neighbor* yang lebih buruk sudah di bawah 0,05 setelah sekitar iterasi 650 (saat T sekitar 2) pada kedua dataset, padahal pencarian berlangsung hingga 1.700 iterasi. Dengan kata lain sekitar 60% iterasi terakhir berjalan dengan perilaku yang setara *stochastic hill climbing*. Hanya 54 sampai 150 dari 1.700 iterasi per *run* yang tercatat sebagai *neighbor* lebih buruk. Sebagian besar *successor* acak bernilai sama atau lebih baik, atau ditolak oleh *constraint*.

Frekuensi *stuck per run* adalah 11 sampai 12 pada `satu_truk` dan 7 pada `multi_truk`, dari maksimum teoretis 17 kejadian (1.700 iterasi dibagi `stuck_limit` 100). Sekitar 65% sampai 71% durasi pencarian pada `satu_truk` dan 41% pada `multi_truk` dihabiskan pada nilai yang tidak berubah. Pada semua *run*, nilai terbaik selama pencarian sama dengan nilai akhir, sehingga SA tidak kehilangan solusi terbaik yang pernah dicapainya.

Dari sisi kualitas, SA memberi hasil terendah di antara seluruh algoritma pada `satu_truk` (557, lebih rendah daripada *stochastic hill climbing* yang bernilai 592) dan hanya mengungguli *stochastic hill climbing* pada `multi_truk` (1.150 dibanding 1.062), seperti terlihat pada **Tabel 2.5.1**. Dua penyebab yang mungkin adalah peluang penerimaan yang tidak sebanding dengan skala selisih nilai (satu paket dapat bernilai puluhan poin sehingga penurunan satu paket langsung memberi $e^{\Delta E/T}$ kecil pada T sedang). Penyebab lain adalah lemahnya sinyal pada plateau, neighbor bernilai sama selalu diterima ($e^{0} = 1$) sehingga SA berpindah-pindah acak di plateau tanpa arah menuju paket tambahan. Parameter SA tidak di-*fine tune* pada eksperimen ini, sehingga hasil ini belum menunjukkan batas kemampuan SA.

2.4. Implementasi *Genetic Algorithm*

Genetic Algorithm mengikuti kerangka pada Russell (2020). Perbedaan penting terhadap kerangka tersebut disebabkan oleh sifat persoalan, yaitu gabungan gen kedua induk hampir selalu menghasilkan penempatan yang melanggar *constraint*, sehingga perlu diperbaiki (*repair*).

Satu individu pada GA direpresentasikan satu objek `State`, dan *fitness*-nya adalah nilai objektif `State`. Populasi awal dibangkitkan dengan `State.random_initial` sebanyak `population_size` kali, dan *state* awal yang digunakan adalah individu terbaik pada populasi awal tersebut. Pada tiap generasi, induk dipilih dengan *roulette wheel*, yaitu peluang terpilih sebanding dengan *fitness* (`random.choices` berbobot), dan induk dipilih seragam jika seluruh *fitness* bernilai 0. Dengan peluang `mutation_rate`, anak yang dihasilkan digantikan oleh satu *successor* acak dari `random_successor`, yaitu satu dari ketiga jenis *move* persoalan. Sebanyak `elite_count` individu terbaik disalin langsung ke generasi berikutnya sehingga nilai maksimum populasi tidak pernah menurun.

Untuk *crossover*, metode `to_chromosome` pada kelas GA mengubah `State` menjadi kromosom berupa daftar gen satu per paket, masing-masing berisi (truk, posisi, orientasi) (**Kode 2.4.1**). Dua induk menghasilkan satu anak melalui *crossover* satu titik potong acak, dengan gen sebelum titik potong berasal dari induk pertama dan sisanya dari induk kedua. Anak dibangun kembali oleh `from_chromosome` dengan prosedur *repair*. Seluruh orientasi dipasang terlebih dahulu, lalu paket dimasukkan berurutan dari z terendah dan setiap paket yang melanggar *constraint* dikeluarkan kembali (**Kode 2.4.2**). Urutan menurut z menjamin paket penopang diproses sebelum paket yang ditopang.

Kode 2.4.1. Konversi *state* menjadi kromosom.

```python
@staticmethod
def to_chromosome(state: State) -> list[Gene]:
    chromosome = []
    for package_id in state.packages:
        chromosome.append((state.truck_of[package_id],
                           state.positions[package_id],
                           state.orientations[package_id]))
    return chromosome
```

Kode 2.4.2. Pembentukan anak dari kromosom beserta *repair*.

```python
def from_chromosome(self, chromosome: list[Gene]) -> State:
    state = State(self.problem)
    all_ids = list(state.packages)
    for package_id, (_, _, orientation) in zip(all_ids, chromosome):
        state.orientations[package_id] = orientation

    # Repair
    insertion_order = []
    for package_id, (truck_index, position, _) in zip(all_ids, chromosome):
        if truck_index is not None and position is not None:
            insertion_order.append((position[2], package_id,
                                    truck_index, position))
    insertion_order.sort()

    dropped = []
    for _, package_id, truck_index, position in insertion_order:
        state.place(package_id, truck_index, position)
        if not state.valid_after_change([package_id]):
            state.place(package_id, None, None)
            dropped.append(package_id)

    if self.reinsert and dropped:
        self.reinsert_dropped(state, sorted(dropped, key=all_ids.index))
    return state
```

Paket yang terbuang saat *repair* kemudian dicoba dimasukkan kembali oleh heuristik *reinsertion* ke posisi layak terendah (urutan z, lalu x, lalu y) pada truk pertama yang masih memuatnya, dengan orientasi tetap (**Kode 2.4.3**). Pencarian posisi memakai `OccupancyGrid`, yaitu grid okupansi tiga dimensi per truk dengan jumlah-prefiks dua dimensi, sehingga seluruh posisi pada satu ketinggian diperiksa sekaligus tanpa mengevaluasi kandidat satu per satu. Heuristik ini merupakan rancangan sendiri dan dapat dimatikan melalui parameter `reinsert` untuk ablasi.

Kode 2.4.3. Heuristik *reinsertion* paket yang terbuang.

```
@staticmethod
def reinsert_dropped(state: State, package_ids: list[str]) -> None:
   # Heuristik repair
   grids = [OccupancyGrid(state, truck_index) for truck_index in
           range(len(state.trucks))]
   for package_id in package_ids:
       package = state.packages[package_id]
       dx, dy, dz = package.oriented_dimensions(state.orientations[package_id])
       for truck_index, truck in enumerate(state.trucks):
           if state.total_weight(truck_index) + package.weight >
              truck.max_capacity:
               continue
           position = grids[truck_index].lowest_position(dx, dy, dz,
                                                         package.is_fragile)
           if position is not None:
               state.place(package_id, truck_index, position)
               grids[truck_index].fill(state.box(package_id), package.is_fragile)
               break
```

Satu iterasi adalah satu generasi. Sebelum dan sesudah tiap generasi, nilai maksimum dan rata-rata populasi dicatat untuk diplot. Setelah `num_iterations` generasi, individu terbaik pada populasi akhir dikembalikan.

Kode 2.4.4. Pencarian *Genetic Algorithm*.

```
def search(self) -> SearchResult:
    population = [State.random_initial(self.problem) for _ in
                  range(self.population_size)]
    initial_state = max(population, key=State.objective)

    history_max = []
    history_avg = []

    for generation in range(self.num_iterations + 1):
        fitness = [individual.objective() for individual in population]
        history_max.append(max(fitness))
        history_avg.append(sum(fitness) / self.population_size)
        if generation == self.num_iterations:
            break

        # Elitisme, individu terbaik langsung dibawa ke generasi berikutnya
        order = sorted(range(self.population_size), key=lambda index:
                       fitness[index], reverse=True)
        new_population = [population[index] for index in
```

```
                    order[:self.elite_count]]

        while len(new_population) < self.population_size:
            parent_1 = self.select(population, fitness)
            parent_2 = self.select(population, fitness)
            child = self.reproduce(parent_1, parent_2)
            if random.random() < self.mutation_rate:
                child = child.random_successor()
            new_population.append(child)
        population = new_population

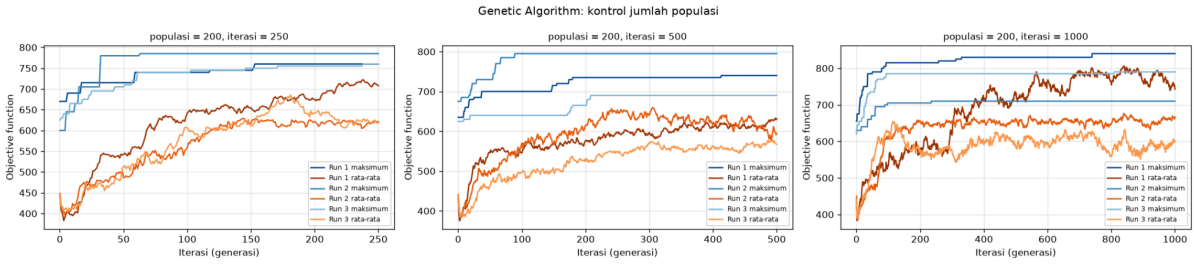
    return SearchResult(initial_state, max(population, key=State.objective),
                        history_max,iterations=self.num_iterations,
                        info={"history_avg": history_avg})
```

Tabel 2.4.1 memuat hasil GA pada kedua skenario kontrol, dan **Gambar 2.4.1** sampai **Gambar 2.4.4** memuat kurva nilai maksimum dan rata-rata populasi per generasi.

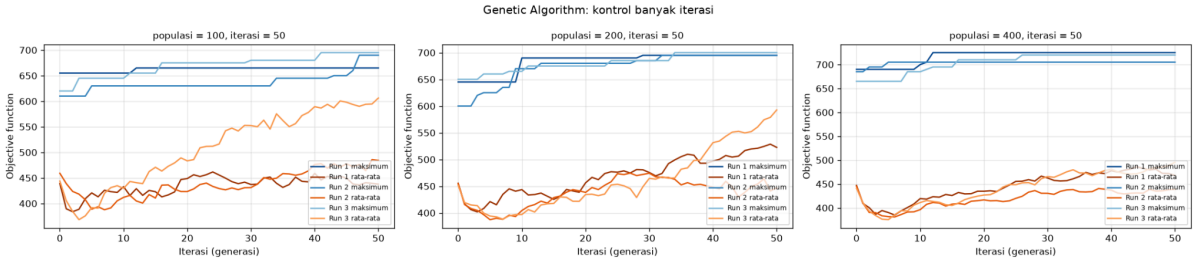
Tabel 2.4.1. Hasil GA pada `satu_truk` dan `multi_truk` pada kedua skenario kontrol.

Skenario	***Objective Function***				**Rasio UB**	**Iterasi**	**Durasi (s)**
	Awal	**Rerata**	**SD**	**Terbaik**			
`satu_truk`, kontrol populasi = 200							
Iterasi 250	632	768,3	14,4	785	84,9%	250	5,71
Iterasi 500	645	741,7	52,5	795	82,0%	500	12,85
Iterasi 1.000	635	780,0	65,6	840	86,2%	1000	24,82
satu_truk, kontrol iterasi = 50							
Populasi 100	628	683,3	16,1	695	75,5%	50	0,89
Populasi 200	632	696,7	2,9	700	77,0%	50	1,76
Populasi 400	680	716,7	10,4	725	79,2%	50	4,25
`multi_truk`, kontrol populasi = 200							
Iterasi 250	628	683,3	16,1	695	75,5%	50	0,89
Iterasi 500	632	696,7	2,9	700	77,0%	50	1,76
Iterasi 1.000	680	716,7	10,4	725	79,2%	50	4,25
`multi_truk`, kontrol iterasi = 50							

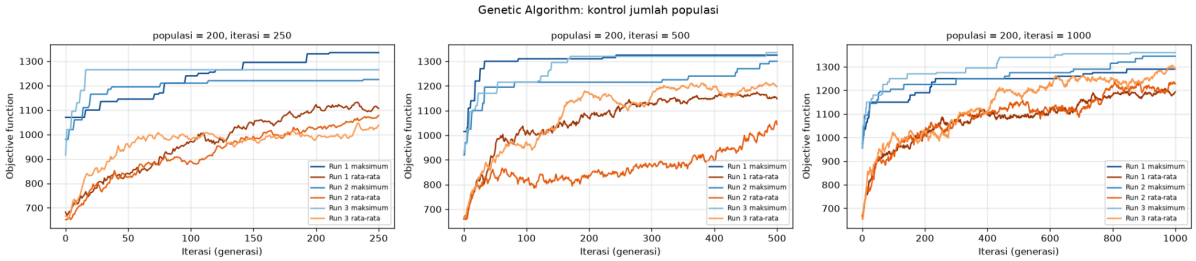
Populasi 100	960	1173,3	38,2	1215	79,8%	50	1,92
Populasi 200	1002	1218,3	18,9	1240	82,9%	50	4,97
Populasi 400	972	1246,7	14,4	1255	84,8%	50	9,10



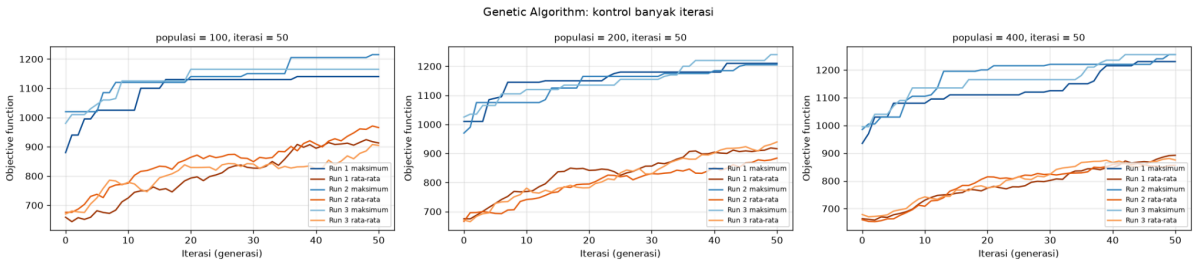
Gambar 2.4.1. Nilai maksimum dan rata-rata populasi pada `satu_truk`, populasi 200 dengan iterasi 250, 500, dan 1.000.



Gambar 2.4.2. Nilai maksimum dan rata-rata populasi pada `satu_truk`, iterasi 50 dengan populasi 100, 200, dan 400.



Gambar 2.4.3. Nilai maksimum dan rata-rata populasi pada `multi_truk`, populasi 200 dengan iterasi 250, 500, dan 1.000.



Gambar 2.4.4. Nilai maksimum dan rata-rata populasi pada `multi_truk`, iterasi 50 dengan populasi 100, 200, dan 400.

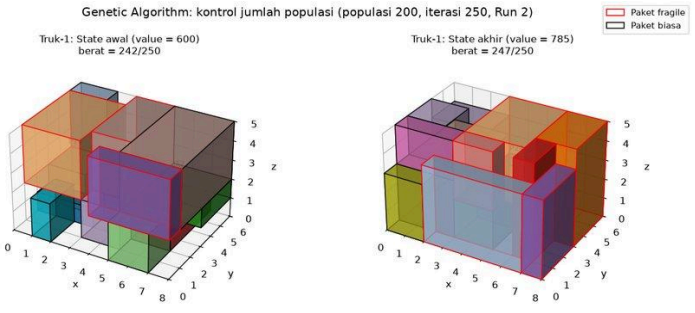
Pengaruh banyak iterasi terlihat pada skenario kontrol populasi. Pada populasi 200, hasil pada `multi_truk` naik seiring iterasi (1.275, 1.320, dan 1.332 untuk 250, 500, dan 1.000 iterasi), dengan

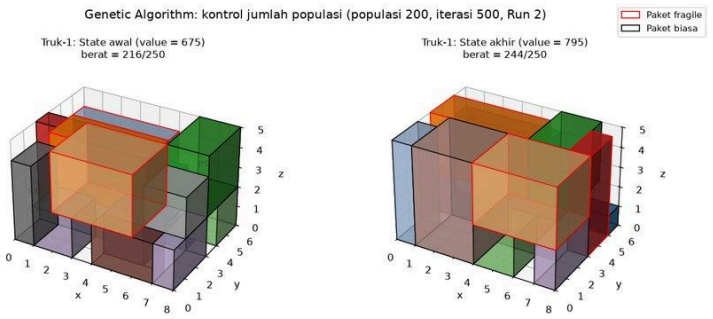
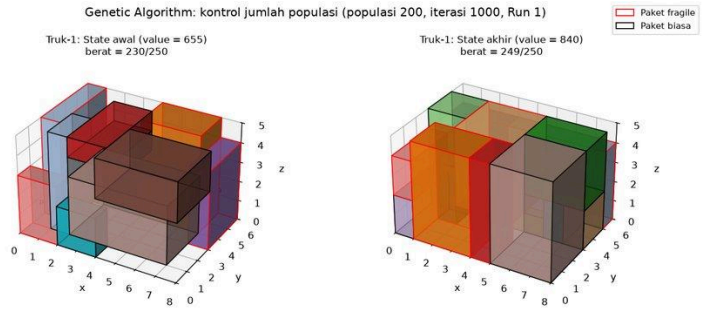
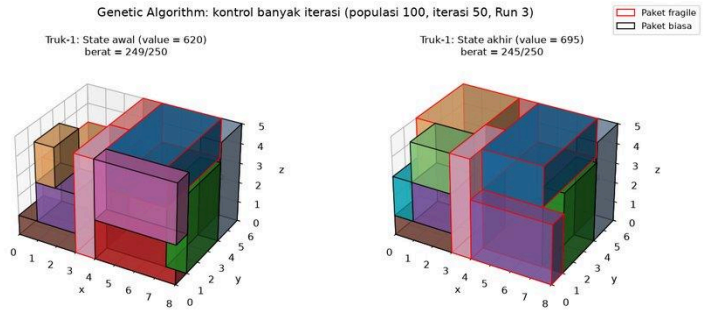
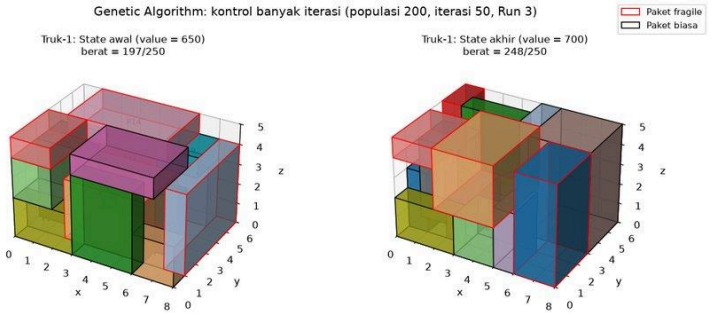
kenaikan yang semakin kecil. Pada `satu_truk` hasilnya tidak monoton (768, 742, 780) dan seluruh selisihnya lebih kecil daripada SD (14,4; 52,5; dan 65,6), sehingga penurunan pada 500 iterasi lebih tepat dibaca sebagai ragam antar-*run* dengan tiga run daripada efek nyata. Durasi bertambah hampir sebanding dengan banyak iterasi pada `satu_truk` (5,7; 12,9; dan 24,8 detik) dan kurang dari sebanding pada `multi_truk` (17,6; 30,5; dan 53,4 detik). Kurva pada **Gambar 2.4.1** menunjukkan nilai maksimum naik bertahap berupa tangga, sedangkan nilai rata-rata naik lebih lancar dan menyusul nilai maksimum secara lambat.

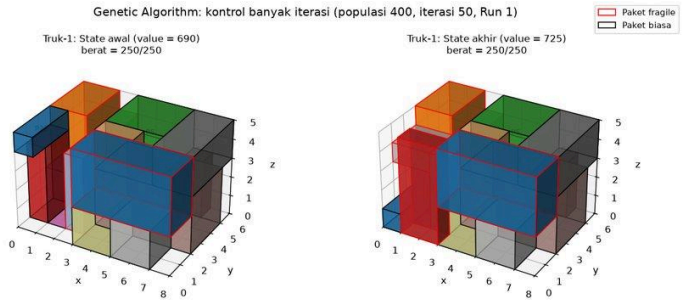
Pengaruh jumlah populasi terlihat pada skenario kontrol iterasi. Pada 50 iterasi, populasi yang lebih besar memperbaiki rerata pada kedua dataset secara konsisten (683, 697, dan 717 pada `satu_truk`, serta 1.173, 1.218, dan 1.247 pada `multi_truk` untuk populasi 100, 200, dan 400). Durasi bertambah sedikit lebih cepat daripada ukuran populasi (0,9; 1,8; dan 4,3 detik pada `satu_truk`). Populasi besar memperbesar peluang adanya individu bernilai tinggi pada populasi awal, yang tercermin pada kolom Awal (628, 632, dan 680 pada `satu_truk`), tetapi kenaikan akhir yang diperoleh dari proses evolusi pada 50 iterasi hanya 37 sampai 65 poin pada `satu_truk`.

Dari sisi konvergensi, pada 50 iterasi nilai rata-rata populasi masih jauh di bawah nilai maksimumnya (misalnya rata-rata 467 berbanding 717 untuk populasi 400 pada `satu_truk`), sehingga keragaman populasi belum menyempit. Pada 1.000 iterasi rata-rata populasi mendekati maksimum (rerata akhir 670 berbanding 780 pada `satu_truk`, dan 1.237 berbanding 1.332 pada `multi_truk`). Menyempitnya keragaman populasi membuat nilai maksimum cenderung mendatar pada generasi akhir, sehingga tambahan generasi memberi manfaat yang makin kecil.

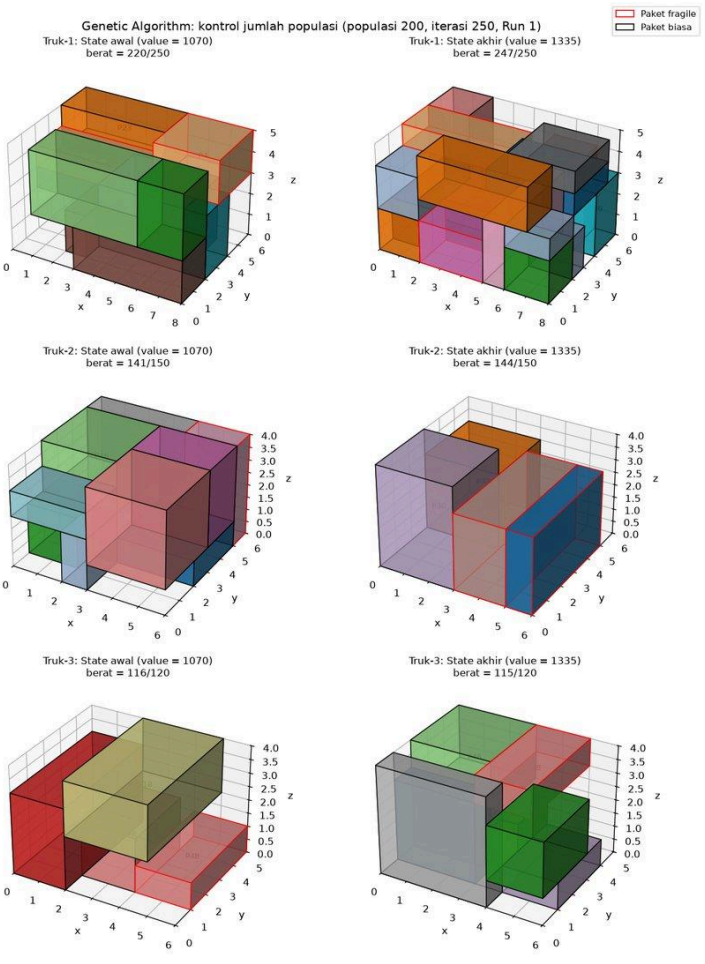
Tabel 2.4.1.1. Visualisasi *state* awal dan akhir GA pada satu_truk.

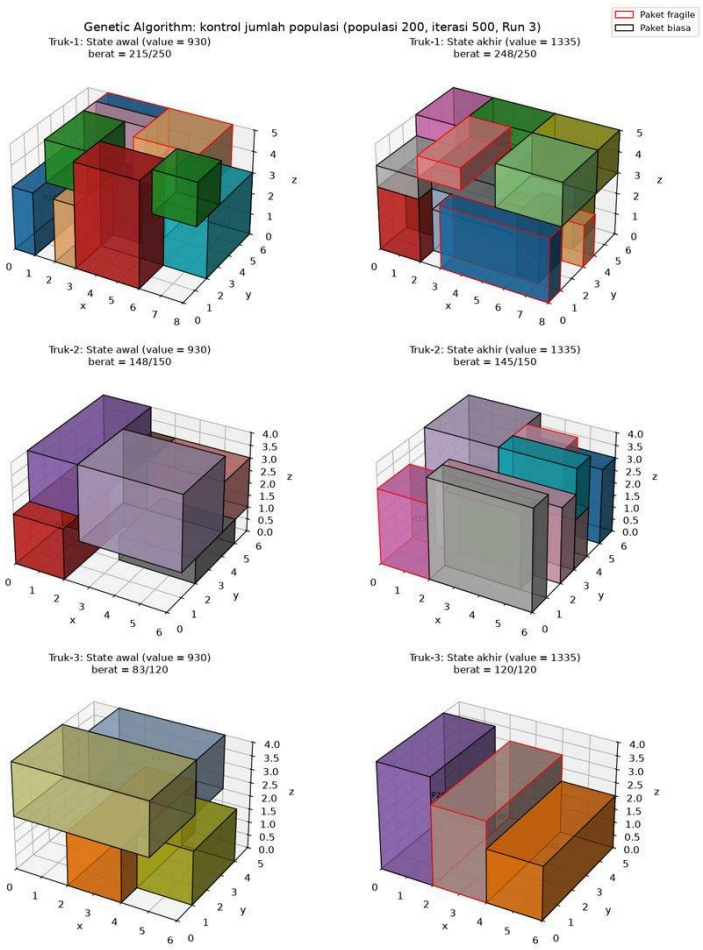
Konfigurasi	Visualisasi Penempatan Paket pada Truk
Populasi 200, iterasi 250 (*run* 2, 600 → 785)	

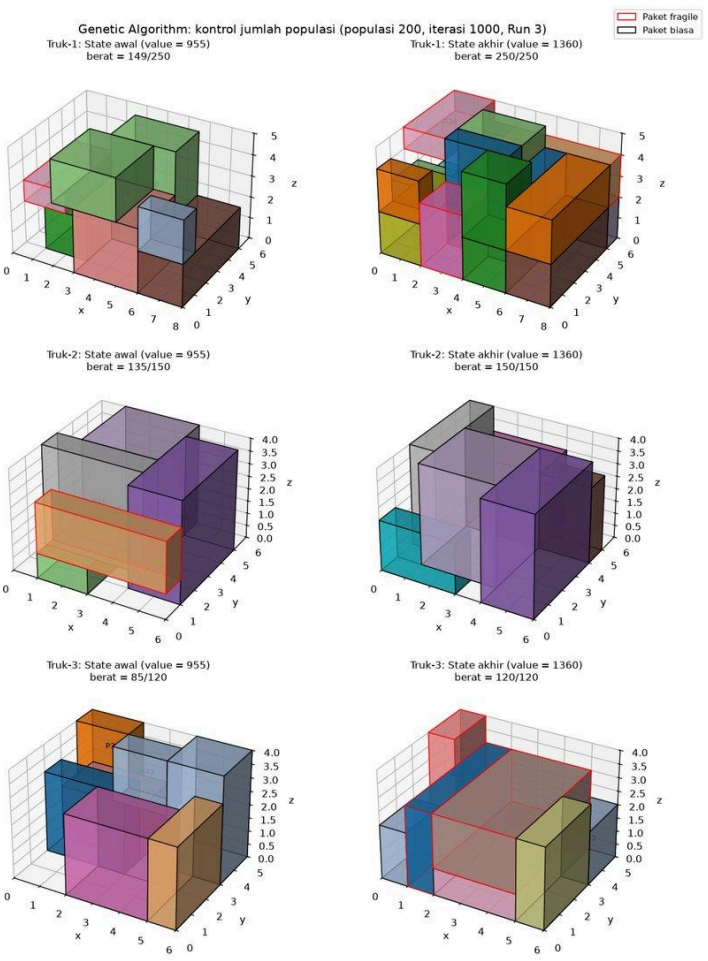
Konfigurasi	Visualisasi Penempatan Paket pada Truk
Populasi 200, iterasi 500 (*run* 2, 675 → 795)	
Populasi 200, iterasi 1.000 (*run* 1, 655 → 840)	
Populasi 100, iterasi 50 (*run* 3, 620 → 695)	
Populasi 200, iterasi 50 (*run* 3, 650 → 700)	

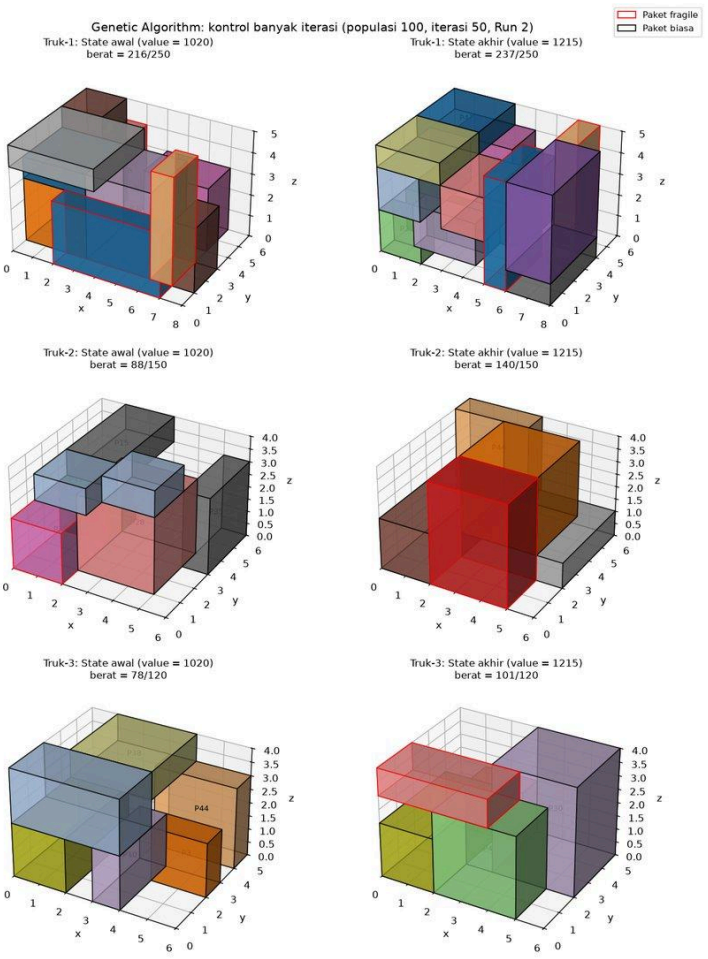
Konfigurasi	Visualisasi Penempatan Paket pada Truk
Populasi 400, iterasi 50 (*run* 1, 690 → 725)	

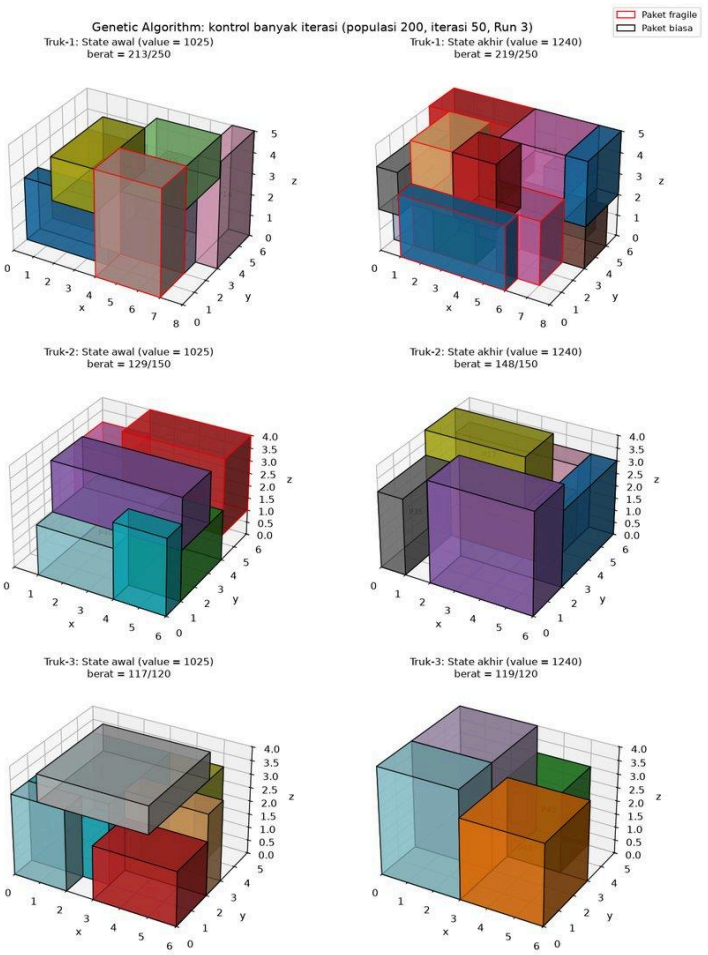
Tabel 2.4.1.2. Visualisasi *state* awal dan akhir GA pada multi_truk.

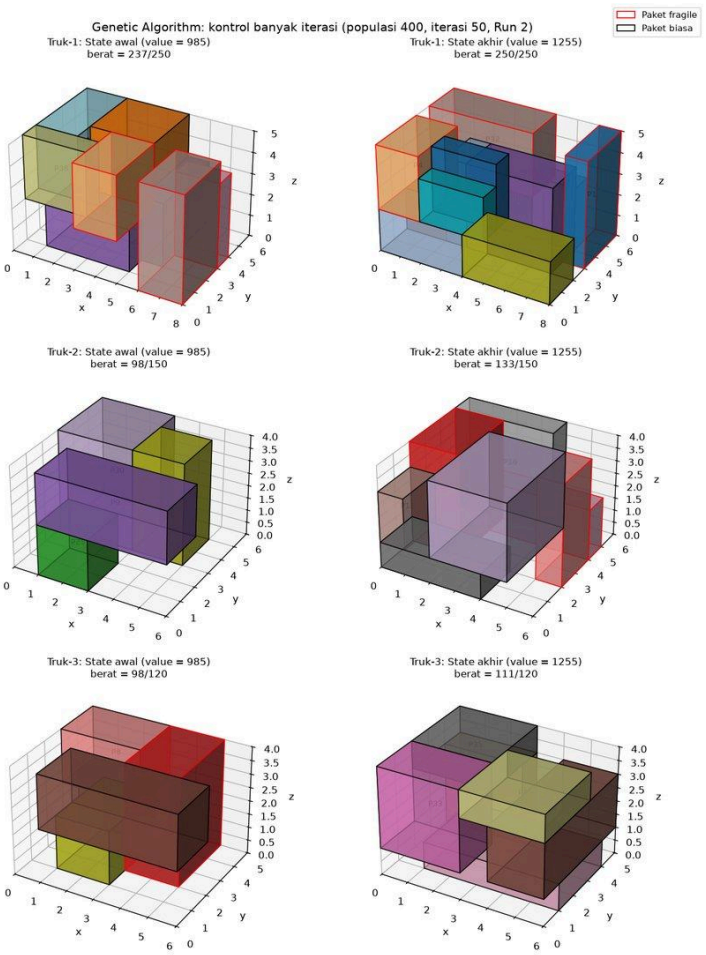
Konfigurasi	Visualisasi Penempatan Paket pada Multi Truk
Populasi 200, iterasi 250 (*run* 1, 1.070 → 1.335)	

Konfigurasi	Visualisasi Penempatan Paket pada Multi Truk
Populasi 200, iterasi 500 (*run* 3, 930 → 1.335)	

Konfigurasi	Visualisasi Penempatan Paket pada Multi Truk
Populasi 200, iterasi 1.000 (*run* 3, 955 → 1.360)	

Konfigurasi	Visualisasi Penempatan Paket pada Multi Truk
Populasi 100, iterasi 50 (*run* 2, 1.020 → 1.215)	

Konfigurasi	Visualisasi Penempatan Paket pada Multi Truk
Populasi 200, iterasi 50 (*run* 3, 1.025 → 1.240)	

Konfigurasi	Visualisasi Penempatan Paket pada Multi Truk
Populasi 400, iterasi 50 (*run* 2, 985 → 1.255)	

Pada state akhir, berat muatan setiap truk mendekati atau sama dengan kapasitasnya. Misalnya pada multi_truk (populasi 200, iterasi 1.000, run 3), berat Truk-1 naik dari 149/250 menjadi 250/250 dan Truk-3 dari 85/120 menjadi 120/120. Pada konfigurasi yang state awalnya sudah penuh, misalnya satu_truk populasi 400 dan iterasi 50 (berat 250/250 pada state awal maupun akhir), kenaikan nilai dari 690 menjadi 725 berasal dari penggantian paket dengan paket yang lebih bernilai per satuan berat, bukan dari penambahan muatan. Seluruh state akhir tetap memenuhi constraint, termasuk paket fragile (garis tepi merah) yang tidak pernah ditumpuki paket lain.

Selain itu, Untuk mengukur pengaruh heuristik *reinsertion* (Kode 2.4.3), GA dengan populasi 200 dan iterasi 250 dijalankan tiga kali dengan reinsert aktif dan tiga kali dengan reinsert nonaktif. Kedua varian memakai *seed* yang sama (67), sehingga populasi awalnya identik dan perbedaan hasil hanya berasal dari proses *repair*. Konfigurasi lain sama dengan Tabel 2.4.1.

Tabel 2.4.2.1. Hasil ablasi heuristik *reinsertion* (populasi 200, iterasi 250).

Dataset	reinsert	Nilai akhir per *run*	Rerata	SD	Durasi rerata (s)
satu_truk	OFF	740, 705, 765	736,7	30,1	4,36
satu_truk	ON	790, 690, 795	758,3	59,2	8,72
multi_truk	OFF	1330, 1295, 1220	1281,7	56,2	6,77
multi_truk	ON	1335, 1225, 1265	1275,0	55,7	19,35

Hasil run multi_truk dengan reinsert aktif identik dengan Tabel 2.4.1 (populasi 200, iterasi 250), sehingga hasil ablasi ini dapat direproduksi. Pada satu_truk, seed yang dipakai berbeda dari eksperimen utama karena eksperimen utama dijalankan berurutan setelah eksperimen lain, sehingga nilainya sedikit berbeda dari Tabel 2.4.1.

Heuristik reinsertion tidak memberi perbaikan yang jelas. Pada satu_truk rerata naik 21,6 poin, tetapi selisih ini lebih kecil daripada SD kedua varian, sedangkan pada multi_truk rerata justru turun 6,7 poin. Di sisi lain, durasi naik sekitar 2 kali pada satu_truk dan hampir 3 kali pada multi_truk, karena setiap anak hasil crossover membangun OccupancyGrid dan mencari posisi untuk setiap paket yang terbuang. Dengan hanya tiga run per varian, kesimpulan yang dapat diambil adalah bahwa pada 250 generasi, crossover, mutasi, dan seleksi sudah cukup untuk mengisi kembali ruang yang kosong akibat repair.

2.5. Perbandingan Antar Algoritma

Untuk GA dipakai dua konfigurasi dari skenario kontrol populasi sebagai wakil, yaitu populasi 200 dengan iterasi 250 (konfigurasi tercepat) dan iterasi 1.000 (konfigurasi terbaik).

Tabel 2.5.1. Perbandingan seluruh algoritma.

Algoritma	Objective Function				Rasio UB	Iterasi	Durasi (s)
	Awal	Rerata	SD	Terbaik			
`satu_truk`							
HC *Steepest Ascent*	508	618,3	43,7	655	68,3%	6	0,17
HC *Sideways Move*	515	658,3	24,7	675	72,7%	26	0,27
HC *Random Restart*	417	676,7	48,6	730	74,8%	52	0,79

Algoritma	Objective Function				Rasio UB	Iterasi	Durasi (s)
	Awal	Rerata	SD	Terbaik			
HC *Stochastic*	498	591,7	5,8	595	65,4%	500	0,02
Simulated Annealing	350	556,7	40,4	600	61,5%	1700	0,10
GA (200, 250)	632	768,3	14,4	785	84,9%	250	5,71
GA (200, 1.000)	635	780,0	65,6	840	86,2%	1000	24,82
`multi_truk`							
HC *Steepest Ascent*	677	1291,7	27,5	1320	87,9%	16	0,38
HC *Sideways Move*	672	1253,3	72,9	1330	85,3%	46	1,24
HC *Random Restart*	567	1343,3	16,1	1355	91,4%	100	2,80
HC *Stochastic*	632	1061,7	29,3	1095	72,2%	500	0,02
Simulated Annealing	677	1160,0	26,0	1190	78,9%	1700	0,15
GA (200, 250)	988	1275,0	55,7	1335	86,7%	250	17,56
GA (200, 1.000)	962	1331,7	36,9	1360	90,6%	1000	53,44

Pada `satu_truk`, urutan rerata nilai dari tertinggi adalah GA (780 dan 768), disusul *Random Restart* (677). Pada `multi_truk`, rerata tertinggi dicapai *Random Restart* (1.343), disusul oleh GA (1.332 untuk iterasi 1.000 dan 1.275 untuk iterasi 250). Algoritma terbaik bergantung pada *dataset* yang digunakan, tetapi secara umum yang paling mungkin mendapatkan hasil terbaik adalah GA dan *Random Restart*, dengan *trade-off* GA berupa waktu komputasi yang besar.

Optimum global sebenarnya tidak diketahui, sehingga kedekatan hasil tiap algoritma diukur terhadap batas atas *U* yang sudah didefinisikan di **1.5. Batas Atas Global Optimum**. Untuk *dataset* yang digunakan, batas atas *U* adalah 905 pada `satu_truk` dan 1.470 pada `multi_truk`. Kedekatan terhadap *U* ditampilkan pada **Tabel 2.5.1** kolom Rasio UB. Pada `satu_truk`, algoritma yang paling mendekati *U* adalah GA dengan rerata mencapai 86,2% dari *U*. Pada `multi_truk` rerata terbaik dicapai oleh Random Restart, sebesar 91,4%. Rasio ini adalah batas bawah dari kedekatan terhadap optimum sebenarnya, karena *U* mengabaikan bentuk paket, penopang, dan pembagian ke tiap truk, sehingga optimum sebenarnya hampir pasti lebih kecil dari *U*.

Hasilnya demikian karena kendala berat menjadi pembatas utama. *State* akhir sebagian besar algoritma sudah memakai 93% sampai 99% kapasitas berat, kecuali *Steepest Ascent* pada `satu_truk` dan *Stochastic* (80% sampai 89%), sehingga selisih antar algoritma ditentukan oleh paket mana yang dimuat daripada seberapa penuh truk. Setelah truk penuh, paket bernilai rendah hanya dapat diganti paket

bernilai lebih tinggi dalam satu langkah bila `swap` kebetulan muat di posisi yang sama. Selain itu, paket harus dikeluarkan terlebih dahulu sehingga nilai turun sementara, dan langkah ini ditolak oleh algoritma yang hanya menerima perbaikan. Susunan tersebut menjadi *local optimum* tempat *Steepest Ascent* dan *Stochastic* berhenti. *Random Restart* mendekati *U* dengan mencoba beberapa titik awal, sedangkan *crossover* GA menggabungkan himpunan paket dari dua induk sekaligus sehingga dapat melompati penurunan sementara tersebut. SA secara teori dapat menerima langkah yang memburuk. Namun, dengan parameter bawaannya, peluang penerimaannya tidak sebanding dengan skala nilai paket, sehingga hasilnya justru terendah pada `satu_truk`.

Dari sisi biaya komputasi, perbandingan antaralgoritma tidak setara. GA dengan iterasi 1.000 memerlukan 24,8 detik pada `satu_truk` dan 53,4 detik pada `multi_truk`, sedangkan *Random Restart* hanya membutuhkan 0,8 dan 2,8 detik. Pada `satu_truk`, GA unggul dengan selisih 91–103 poin dari *Random Restart* meskipun biayanya 7–31 kali lebih besar. Pada `multi_truk`, GA dengan biaya ~19 kali lebih besar tidak mengungguli *Random Restart*.

Durasi tiap algoritma berskala berbeda tergantung ukuran persoalan. *Stochastic* dan SA menggunakan satu *successor* acak per iterasi, sehingga durasinya tidak berubah ketika paket bertambah dari 36 (`satu_truk`) menjadi 48 (`multi_truk`), memakan waktu 0,02 vs. ~0,09 detik. Algoritma yang membangkitkan seluruh *successor* (*Steepest Ascent, Sideways Move, Random Restart*) menjadi sekitar 2,2–4,6 kali lebih lama pada `multi_truk` karena jumlah *neighbor* per *state* bertambah mengikuti jumlah paket, truk, dan posisi.

2.6. Bonus: Fungsi Objektif Urutan Bongkar (LIFO)

Fungsi objektif yang telah didefinisikan hanya menilai paket mana saja yang dimuat, tetapi tidak menilai bagaimana paket disusun. Fungsi objektif ini mempertimbangkan urutan pengeluaran paket menggunakan atribut ETA, di mana paket dengan ETA lebih kecil diturunkan lebih dahulu. Susunan yang dinilai baik menempatkan paket yang turun lebih awal sedemikian rupa sehingga dapat dikeluarkan tanpa memindahkan paket yang turun belakangan, seperti halnya prinsip *last in, first out* (LIFO).

2.6.1. Definisi

Letak pintu. Pintu truk berada pada ujung sumbu horizontal terpanjang, yaitu bidang $x = W$ jika $W \geq L$ dan bidang $y = 0$ jika $W < L$. Paket dikeluarkan dengan ditarik menuju pintu atau diangkat ke atas.

Relasi menghalangi. Untuk dua paket *i* dan *j* pada truk yang sama dengan kotak $(x,\ y,\ z,\ d_x, d_y, d_z)$, paket *j* menghalangi paket *i* jika salah satu kondisi berikut terpenuhi:

1. *j* berada di atas *i,* yaitu proyeksi alas keduanya pada bidang xy beririsan dan $z_j \geq z_i + d_{z,i}$. Seluruh paket pada tumpukan di atas *i* ikut terhitung, bukan hanya yang bersentuhan langsung.
2. *j* berada di antara *i* dan pintu. Untuk pintu pada $x = W$, proyeksi keduanya pada bidang yz beririsan dan $x_j \geq x_i + d_{x,i}$. Untuk pintu pada $y = 0$, proyeksi pada bidang xz beririsan dan $y_j + d_{y,j} \leq y_i$.

Penalti. Pasangan (*i, j*) melanggar urutan bongkar apabila *j* menghalangi *i* dan $\text{eta}_j > \text{eta}_i$. Pasangan dengan ETA sama tidak dikenakan penalti. Bobot pelanggaran sebanding dengan selisih ETA dan dinormalisasi terhadap rentang ETA pada *dataset*, sehingga pelanggaran terparah bernilai 1.

$$P(s) = \sum_{(i,j)\ \text{melanggar}} \frac{\text{eta}_j - \text{eta}_i}{\text{eta}_{max} - \text{eta}_{min}}$$

Fungsi objektif. Penalti dikurangkan dari total nilai dengan bobot λ (*default* $\lambda = 10$). Fungsi objektif ini bersifat *trade-off*. Algoritma dapat melepas paket dengan *value* kecil jika keberadaannya akan menimbulkan penalti yang lebih mahal.

$$f(s) = \sum_{i\ \text{di dalam truk}} \text{value}_i - \lambda P(s)$$

Mode *tie-breaker.* Sebagai alternatif, penalti dapat dibuat tidak pernah mengalahkan selisih nilai. Karena tiap pasangan yang terhitung penalti menyumbang paling banyak 1 setelah dinormalisasi, $P(s) \leq n(n-1)/2$ untuk n paket, sehingga dengan

$$f(s) = \sum_{i\ \text{di dalam truk}} \text{value}_i - \frac{P(s)}{n(n-1)/2 + 1},$$

total penalti selalu kurang dari 1. Karena seluruh *value* adalah bilangan bulat, urutan *state* menurut f sama dengan urutan *state* menurut nilai, dan penalti hanya membedakan *state* yang nilainya sama.

2.6.2. Implementasi

Objektif dipilih melalui parameter `objective`, `penalty_weight`, dan `tie_breaker` pada menu parameter. `Problem.configure_objective` menerjemahkan ketiganya menjadi satu faktor skala sehingga `State.objective` cukup menghitugn value − skala × penalti.

Kode 2.6.2.1. Konfigurasi faktor skala penalti dan penghitungan fungsi objektif.

```
def configure_objective(
    self, objective: str, penalty_weight: float, tie_breaker: bool
) -> None:
    # objective "value" = total value
    # objective "lifo" = total value - penalty
    # penalty_scale mengubah raw penalty (jumlah selisih ETA) menjadi pengurang
objective
    self.objective = objective
    self.penalty_weight = penalty_weight
    self.tie_breaker = tie_breaker
    if objective == "value" or self.eta_range == 0:
        self.penalty_scale = 0.0
    elif tie_breaker:
        # Tiap pasangan menyumbang max 1 setelah dinormalisasi, total penalty < 1
        num_packages = len(self.package_list)
        max_pairs = num_packages * (num_packages - 1) // 2
        self.penalty_scale = 1 / (self.eta_range * (max_pairs + 1))
    else:
        self.penalty_scale = penalty_weight / self.eta_range
```

```
def objective(self) -> float:
    if self.problem.penalty_scale == 0:
        return self.value
    return self.value - self.problem.penalty_scale * self.penalty
```

Relasi menghalangi diimplementasikan pada fungsi `blocks`, dan kontribusi satu pasanagan pada `State.pair_penalty`.

Kode 2.6.2.2. Relasi menghalangi terhadap tumpukan dan arah pintu dan penghitungan penalti.

```
def blocks(blocker: Box, target: Box, door_on_x: bool) -> bool:
    bx, by, bz, bdx, bdy, bdz = blocker
    tx, ty, tz, tdx, tdy, tdz = target
```

```
    if bz >= tz + tdz and bases_overlap(blocker, target):
        return True
    if not intervals_overlap(bz, bdz, tz, tdz):
        return False
    if door_on_x:
        return bx >= tx + tdx and intervals_overlap(by, bdy, ty, tdy)
    return by + bdy <= ty and intervals_overlap(bx, bdx, tx, tdx)
```

```
@staticmethod
def pair_penalty(
    box_1: Box, eta_1: int, box_2: Box, eta_2: int, door_on_x: bool
) -> int:
    if eta_1 < eta_2 and blocks(box_2, box_1, door_on_x):
        return eta_2 - eta_1
    if eta_2 < eta_1 and blocks(box_1, box_2, door_on_x):
        return eta_1 - eta_2
    return 0
```

Penghitungan penalti secara langsung memiliki biaya $O(n^2)$ per *state*, yang sangat tidak efisien pada algoritma yang menghasilkan semua *successor* per iterasi seperti *Steepest Ascent*. Untuk menanggulanginya, penalti disimpan sebagai *cache* `penalty`. Setiap kali `place` atau `rotate` mengubah paket k, kontribusi seluruh pasangan yang melibatkan k dikurangkan seblum perubahan dan dihitung sesudahnya (`package_penalty`), sehingga biayanya $O(n)$ per perubahan.

Berbeda dengan objektif `value` yang selalu berupa *integer*, objektif `lifo` bernilai pecahan (*float*). Jika *cache* menyimpan penalti yang sudah dinormalisasi, nilainya diperbarui lewat rangkaian penjumlahan dan pengurangan pecahan yang *error* pembulatannya bergantung pada urutan operasi. Dua *state* yang sama tetapi dicapai lewat jalur berbeda dapat memiliki objektif yang berselisih sangat kecil, sehingga perbandingan kesamaan nilai pada *Sideways Move* dan pendeteksian *stuck* pada SA menjadi tidak tepat. Oleh karena itu, *cache* menyimpan penalti mentah berupa jumlah selisih ETA (*integer*) dan baru dinormalisasi ketika objektif dibaca.

Kode 2.6.2.2. Penalti inkremental satu paket.

```
def package_penalty(self, package_id: str) -> int:
    # Total penalty semua pasangan yg ngelibatin package_id di truk yg sama
    if self.problem.objective != "lifo" or package_id not in self.boxes:
        return 0
    package_box = self.boxes[package_id]
    eta = self.packages[package_id].eta
    door_on_x = self.trucks[self.truck_index_of(package_id)].door_on_x
    total = 0
```

```
    for other_id, other_box in self.other_boxes_in_truck(package_id):
        total += self.pair_penalty(
            package_box, eta, other_box, self.packages[other_id].eta, door_on_x
        )
    return total
```

Pada GA, seleksi *roulette wheel* memakai nilai objektif sebagai bobot, sedangkan f dapat bernilai negatif. Jika terdapat *fitness* negatif, seluruh *fitness* populasi digeser hingga nilai mininumnya 0. Pada objektif `value`, tidak ada *fitness* negatif, sehingga perilaku GA tidak berubah.

Pada visualisasi, bidang pintu diarsir hijau. Ketika objektif `lifo` aktif, paket diwarnai dengan gradasi biru, dari muda untuk ETA terkecil hingga tua untuk ETA terbesar. Susunan yang baik tampak sebagai paket biru muda yang dekat dengan pintu dan berada di puncak tumpukan.

2.6.3. Hasil dan Analisis

Tabel 2.6.3.1 membandingkan rerata nilai akhir (*Value*), penalti akhir (P), dan objektif f dari tiga *run*. Untuk GA, diambil perwakilan seperti pada **2.5. Perbandingan Antar Algoritma**. Kolom f untuk mode tie-breaker tidak ditampilkan karena penaltinya diskalakan sangat kecil, sehingga sama dengan Value setelah dibulatkan ke satu desimal.

Tabel 2.6.3.1. Nilai dan penalti akhir untuk ketiga fungsi objektif.

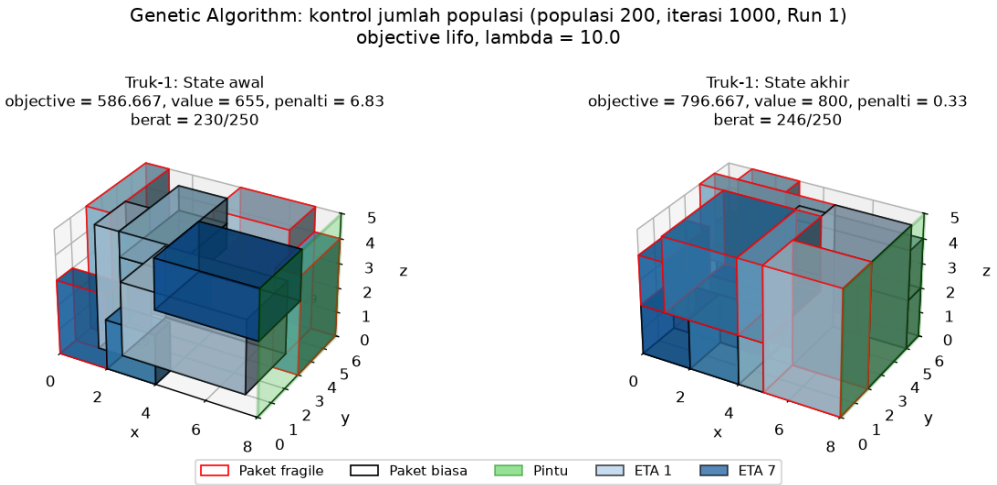
Algoritma	**Objektif `value`**		**`lifo`, $\lambda = 10$**			**`lifo`, *tie-breaker***	
	Value	***P***	***Value***	***P***	***f***	***Value***	***P***
`satu_truk`							
HC *Steepest Ascent*	618,3	13,72	618,3	5,94	558,9	678,3	9,78
HC *Sideways Move*	658,3	9,56	678,3	3,17	646,7	660,0	5,06
HC *Random Restart*	676,7	10,28	710,0	3,78	672,2	743,3	7,28
HC *Stochastic*	591,7	6,78	550,0	2,22	527,8	570,0	3,61
Simulated Annealing	556,7	9,28	603,3	1,89	584,4	573,3	13,94
GA (200, 250)	768,3	7,39	743,3	1,44	728,9	803,3	6,11
GA (200, 1.000)	780,0	10,56	795,0	0,61	788,9	765,0	5,28
`multi_truk`							
HC *Steepest Ascent*	1291,7	11,67	1320,0	3,83	1281,7	1296,7	5,50
HC *Sideways Move*	1253,3	17,33	1266,7	5,11	1215,6	1270,0	7,56

Algoritma	Objektif `value`		`lifo`, $\lambda = 10$			`lifo`, *tie-breaker*	
	Value	***P***	***Value***	***P***	***f***	***Value***	***P***
`multi_truk`							
HC *Random Restart*	1343,3	12,94	1366,7	3,06	1336,1	1380,0	9,11
HC *Stochastic*	1061,7	14,78	1091,7	6,00	1031,7	1075,0	8,94
Simulated Annealing	1160,0	15,06	1170,0	3,11	1138,9	1136,7	12,06
GA (200, 250)	1275,0	16,22	1260,0	5,94	1200,6	1266,7	8,00
GA (200, 1.000)	1331,7	13,78	1310,0	4,06	1269,4	1310,0	13,28

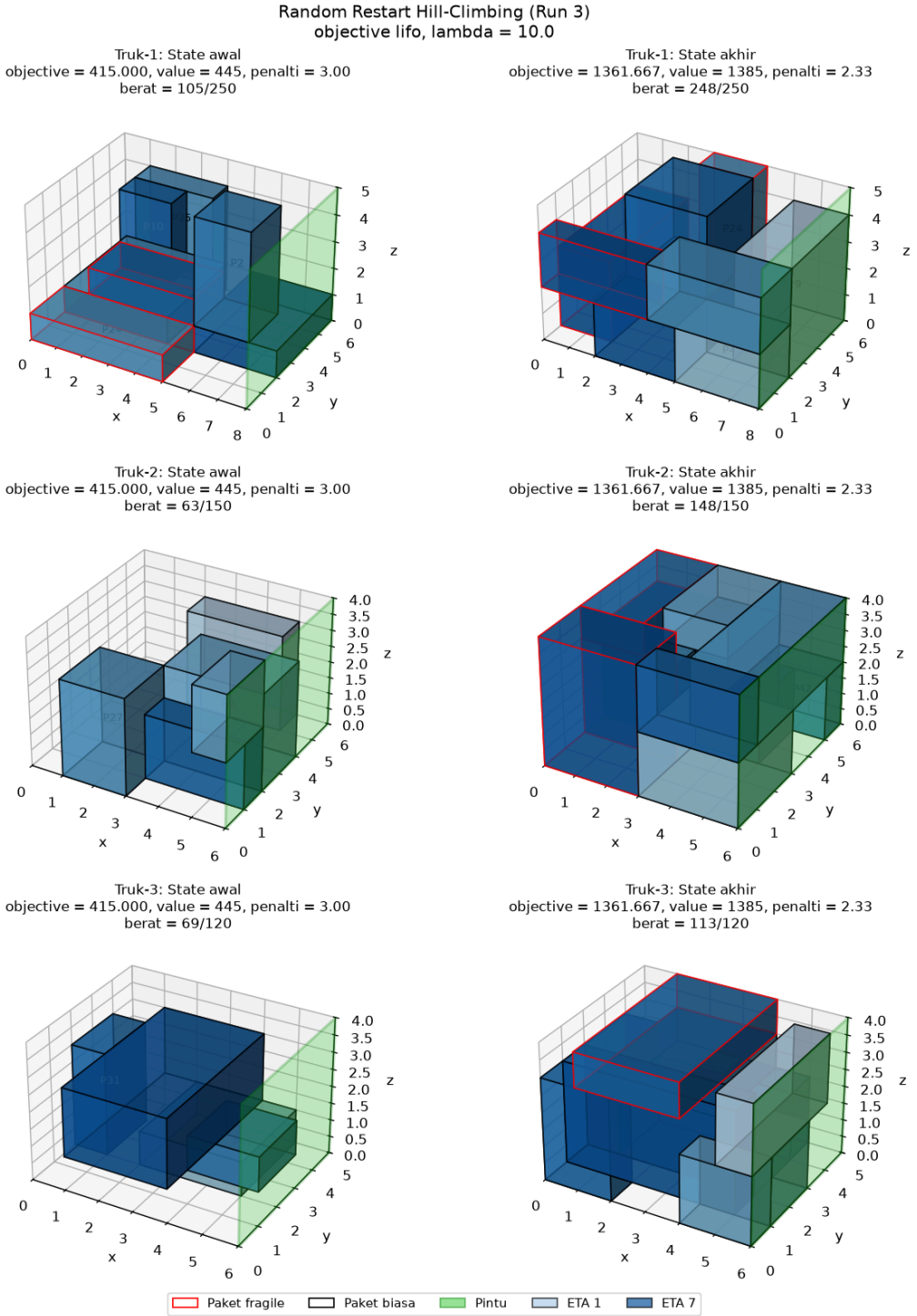
Objektif `value` menghasilkan susunan dengan urutan pembongkaran yang buruk. Karena posisi paket tidak memengaruhi nilai, penalti rerata mencapai 9,65 pada `satu_truk` dan 14,54 pada `multi_truk`, yang bahkan lebih tinggi dari state awalnya (6,16 dan 5,29). Hal ini disebabkan karena makin banyak paket yang dimuat, makin banyak pasangan yang saling menghalangi.

Dengan penalti $\lambda = 10$, penalti turun tajam tanpa terlalu mengorbankan nilai. Penalti per baris turun 57–94% pada `satu_truk` dan 59–79% pada `multi_truk`, sedangkan rerata nilai tidak turun jauh (671,2 vs. 664,3; dan 1.255,0 vs. 1.245,2). Ruang *state* yang didominasi *plateau* memiliki banyak susunan dengan nilai identik, sehingga penalti cukup memilih yang lebih rapi di antaranya. Hasil terbaik diperoleh GA (200, 1.000) pada `satu_truk` dengan rerata nilai 795 dan penalti 0,61.

Gambar 2.6.3.1 dan **Gambar 2.6.3.2** menunjukkan hasil run dengan *f* tertinggi. Pada `satu_truk`, GA menaikkan nilai dari 655 menjadi 800 dan menurunkan penalti dari 6,83 menjadi 0,33. Pada `multi_truk`, *Random Restart* mencapai nilai 1.385 dengan penalti 2,33.



Gambar 2.6.3.1. Run `satu_truk` dengan *f* tertinggi pada objective `lifo`.



Gambar 2.6.3.1. Run `multi_truk` dengan *f* tertinggi pada ovjective `lifo`.

Mode *tie-breaker* hanya mengurangi penalti secara terbatas. Penalti rerata turun 24% dan 37% (menjadi 7,29 dan 9,21) untuk masing-masing `satu_truk` dan `multi_truk`. Nilai tetap sebanding dengan objektif `value`, karena *move* yang menaikkan nilai selalu diambil meskipun menambah pelanggaran.

Penalti memberikan arah pada *plateau*. Banyak *successor* yang semula bernilai sama kini menjadi lebih baik atau buruk, sehingga *hill climbing* mendaki lebih lama sebelum berhenti. Pada mode *tie-breaker*, nilai *Steepest Ascent* dan *Random Restart* juga cenderung naik, karena penalti bekerja seperti *sideways move* yang terarah.

Tabel 2.6.3.2. Rerata iterasi dan durasi untuk ketiga fungsi objektif.

Algoritma	Iterasi			Durasi (s)		
	`value`	$\lambda = 10$	***tie-breaker***	`value`	$\lambda = 10$	***tie-breaker***
`satu_truk`						
HC *Steepest Ascent*	6	10	17	0,17	0,47	0,98
HC *Sideways Move*	26	47	44	0,27	2,47	2,09
HC *Random Restart*	52	72	89	0,79	3,58	4,65
HC *Stochastic*	500	500	500	0,02	0,06	0,05
Simulated Annealing	1700	1700	1700	0,10	0,26	0,28
GA (200, 250)	250	250	250	5,71	10,38	10,68
GA (200, 1.000)	1000	1000	1000	24,82	37,39	38,13
`multi_truk`						
HC *Steepest Ascent*	16	23	28	0,38	1,59	2,02
HC *Sideways Move*	46	63	52	1,24	5,87	5,11
HC *Random Restart*	100	136	134	2,80	11,53	10,39
HC *Stochastic*	500	500	500	0,02	0,07	0,04
Simulated Annealing	1700	1700	1700	0,16	0,26	0,19
GA (200, 250)	250	250	250	17,56	24,48	24,19
GA (200, 1.000)	1000	1000	1000	53,44	77,64	75,01

Biaya komputasi naik karena setiap `place` dan `rotate` menghitung ulang penalti terhadap paket lain di truk yang sama. Durasi naik 1,4–1,9 kali pada GA dan ~3 kali pada *Stochastic* dan SA. Kenaikan terbesar terjadi pada *hill climbing* yang membangkitkan semua successor, sebesar 2,8–9,1 kali, karena biaya per iterasi dan banyak iterasi bertambah.

Secara keseluruhan, objektif `lifo` dengan $\lambda = 10$ menghasilkan susunan yang lebih mudah dibongkar dengan nilai yang setara. Mode *tie-breaker* cocok bila nilai harus tetap menjadi prioritas utama, tetapi pengaruhnya cukup terbatas.

3. Kesimpulan dan Saran

3.1. Kesimpulan

Berdasarkan implementasi dan eksperimen yang telah dilakukan, diperoleh kesimpulan sebagai berikut.

1. Keenam algoritma *local search* (*steepest ascent*, *sideways move*, *random restart*, dan *stochastic hill climbing*, *simulated annealing*, serta *genetic algorithm*) berhasil diimplementasikan pada persoalan pengisian paket ke peti kemas, baik untuk satu truk maupun multi-truk. Dengan representasi *state* yang memetakan setiap paket ke truk, posisi, dan orientasinya, serta pemeriksaan *constraint* inkremental, seluruh algoritma hanya bergerak di antara *state* yang memenuhi kelima *constraint*.

2. Tidak ada algoritma yang selalu terbaik. Pada satu_truk, GA memberi rerata tertinggi (780,0 atau 86,2% dari batas atas *U*), disusul *Random Restart* (676,7 atau 74,8%). Pada multi_truk, *Random Restart* memberi rerata tertinggi (1.343,3 atau 91,4% dari *U*) dengan GA (200, 1.000) di posisi kedua (1.331,7 atau 90,6%). Karena *U* adalah batas atas yang longgar, rasio terhadap optimum sebenarnya lebih tinggi daripada angka-angka ini.

3. Algoritma yang hanya menerima perbaikan (*steepest ascent* dan *stochastic*) paling cepat berhenti di *local optimum*. Penyebab utamanya adalah truk yang sudah hampir penuh secara berat, sehingga mengganti paket bernilai rendah dengan paket bernilai tinggi memerlukan langkah yang menurunkan nilai sementara. Mekanisme diversifikasi berupa *restart* (*random restart*) dan penggabungan dua solusi (*crossover* pada GA) paling efektif mengatasi hal ini.

4. *Sideways move* tidak memberi kenaikan sama sekali pada satu_truk, tetapi membantu dua dari tiga *run* pada multi_truk keluar dari *plateau*. Manfaatnya bergantung pada ada tidaknya ruang untuk menggeser paket.

5. *Simulated annealing* dengan parameter bawaan ($T_0 = 50,\ \alpha = 0,995,\ T_{min} = 0,01$) memberi hasil terendah pada satu_truk (556,7). Peluang penerimaan *neighbor* yang lebih buruk sudah di bawah 0,05 setelah sekitar iterasi 410 sampai 670 dari 1.700, sehingga sebagian besar pencarian berperilaku seperti *stochastic hill climbing*.

6. Dari sisi durasi, *stochastic* dan SA paling cepat (di bawah 0,2 detik), varian *hill climbing* yang membangkitkan seluruh *neighbor* berada di kisaran 0,17 sampai 2,8 detik, sedangkan GA paling lambat (5,7 sampai 53,4 detik). Biaya GA tidak selalu sebanding dengan hasilnya, misalnya pada multi_truk GA (200, 1.000) memerlukan sekitar 19 kali durasi *random restart* tetapi tidak mengunggulinya.

7. Pada GA, menambah banyak iterasi memperbaiki hasil dengan kenaikan yang makin kecil pada multi_truk (1.275,0, 1.320,0, dan 1.331,7), sedangkan pada satu_truk perbedaannya masih lebih kecil daripada ragam antar-*run*. Menambah jumlah populasi memperbaiki rerata secara konsisten pada kedua *dataset*, terutama karena populasi awal yang lebih besar memuat individu yang lebih baik. Heuristik *reinsertion* tidak memberi perbaikan yang jelas pada ablasi, tetapi menggandakan durasi.

8. Fungsi objektif bonus lifo dengan $\lambda = 10$ menurunkan penalti urutan bongkar sebesar 57% sampai 94% tanpa menurunkan rerata nilai muatan, sehingga susunan paket menjadi lebih mudah dibongkar sesuai ETA.

3.2. Saran

1. Parameter SA perlu disetel ulang, misalnya dengan T_0 yang disesuaikan dengan skala selisih nilai paket (puluhan poin) dan pendinginan yang lebih lambat, agar fase eksplorasi tidak berhenti terlalu dini.

2. Jumlah *run* per konfigurasi perlu ditambah (misalnya 10 sampai 30 *run*) agar perbandingan rerata dan SD antar-algoritma dapat diuji secara statistik. Dengan tiga *run*, banyak selisih yang diamati masih lebih kecil daripada ragam antar-*run*.

3. Perbandingan antar-algoritma sebaiknya juga dilakukan pada anggaran waktu atau jumlah evaluasi fungsi objektif yang sama, karena durasi GA dan *random restart* jauh lebih besar daripada *stochastic* dan SA.

4. Batas atas U dapat diperketat, misalnya dengan menambahkan batasan per truk atau batas luas alas, agar rasio terhadap U lebih mendekati rasio terhadap optimum sebenarnya.

5. Langkah *neighbor* dapat diperkaya dengan *move* gabungan seperti mengeluarkan satu paket sekaligus memasukkan paket lain di posisi berbeda, sehingga *hill climbing* dapat keluar dari *local optimum* akibat truk yang sudah penuh tanpa harus menurunkan nilai terlebih dahulu.

Daftar Pustaka

Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

Lampiran

A. Pembagian Tugas dan Repo Github

Tautan repositori GitHub: https://github.com/buege-putra/IF3170_G59_LazyService

NIM	Nama	Pembagian Tugas
10123004	Adiel Rum	*Hill climbing, simulated annealing, genetic algorithm,* Visualisasi paket dan truk.
13323032	Christopher Justin Kurniawan	Class dan atribut dasar, Laporan
13523037	Buege Mahara Putra	Heuristik *genetic algorithm*, TUI dan visualisasi, fungsi objektif LIFO, laporan

B. Input Multi Truk dan Satu Truk

Input file satu-truk dan multi-truk berbeda dengan ekstensi JSON. Dalam satu file JSON berisi banyak truk dan paket beserta variabelnya. Bila case adalah satu truk, maka pada class trucks, hanya ada satu line yaitu "Truk-1". Pada eksperimen digunakan nilai untuk tiap variabel trucks dan packages berbeda antara file input untuk satu truk dan untuk multi truk.

Kode B.1. Contoh input multi-truk berupa file JSON.

```
"trucks": [
    {"id": "Truk-1", "dimensions": {"w": 8, "l": 6, "h": 5},
     "max_capacity": 250},
    {"id": "Truk-2", "dimensions": {"w": 6, "l": 6, "h": 4},
     "max_capacity": 150},
    {"id": "Truk-3", "dimensions": {"w": 6, "l": 5, "h": 4},
     "max_capacity": 120}
],

"packages": [
    {"id": "P1", "dimensions": {"w": 3, "l": 5, "h": 1}, "value": 65,
     "weight": 19, "is_fragile": true, "eta": 6},
    {"id": "P2", "dimensions": {"w": 4, "l": 2, "h": 2}, "value": 60,
     "weight": 13, "is_fragile": false, "eta": 5},
    {"id": "P3", "dimensions": {"w": 2, "l": 1, "h": 2}, "value": 10,
     "weight": 7, "is_fragile": false, "eta": 3},
]
```

C. Bonus yang Dikerjakan

Bonus	Status
Implementasi fungsi objektif lain tanpa menambah variabel	✅
Implementasi semua algoritma hill climbing	✅
Implementasi Multi Truck	✅

Form Penggunaan AI

Dengan ini, kami:

Nama/NIM : Adiel Rum (10123004), Christopher Justin Kurniawan (13323032), Buege Mahara Putra (13523037)
Fakultas/Sekolah : Sekolah Tinggi Elekro dan Informatika
Kode Mata Kuliah : IF3170
Nama Mata Kuliah : Inteligensi Artifisial
Judul Tugas/Proposal : Tugas Besar 1: Local Search

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	~~Ya~~/Tidak	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	~~Ya~~/Tidak	
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	~~Ya~~/Tidak	
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	~~Ya~~/Tidak	
Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026,





Adiel Rum	Christopher Justin Kurniawan	Buege Mahara Putra
10123004	13323032	13523037